

The derivatives of `SolutionSpaceFeature` included features that measured whether the appropriate spaces were placed into the solution, whether the spaces had the appropriate area and aspect ratio, whether elevators stacked properly, etc.

The neat thing about this is that the loop that traverses the data structure is located in one place. All the scoring features inherit it rather than reimplement it.

Some of the features had to measure characteristics of the portals attached to a space. So we reproduced the pattern and created the class `PortalFeature` derived from `SolutionSpaceFeature`. The implementation of `NewSolutionSpace` within `PortalFeature` looped through all the portals in the `SolutionSpace` argument and called the pure virtual function `NewPortal(const Portal&)`. (See Figure 30-5.)

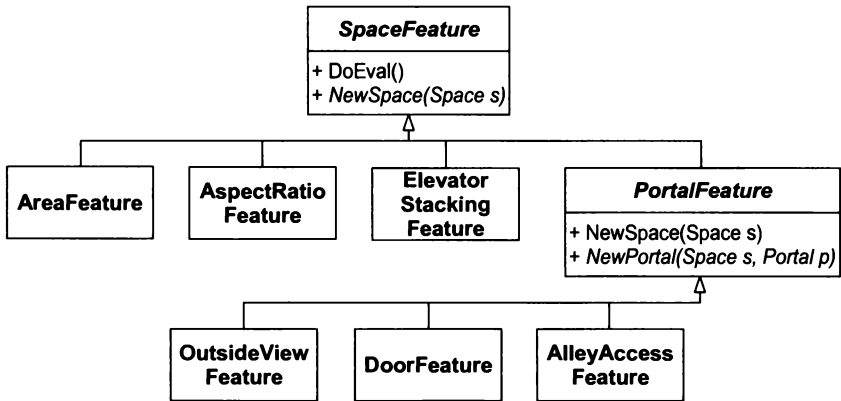


Figure 30-5 TEMPLATE METHOD structure of the scoring features

This structure allowed us to create dozens of different scoring features, each one of which walked the floor-plan data structure, without knowing what the floor-plan data structure looked like. If the details of the floor-plan data structure changed (e.g., we decided to use STL instead of our own iterators), we'd have to change two classes rather than several dozen.

Why did we choose TEMPLATE METHOD over STRATEGY?⁵ Consider how much looser the coupling would have been had we used STRATEGY! (See Figure 30-6.)

Using the TEMPLATE METHOD structure, if we had to make a change to the algorithms that walked the data structure, we would have had to change `SpaceFeature` and `PortalFeature`. In all likelihood, this would have forced us to recompile *all* the features. However, using the STRATEGY pattern, the change would have been restricted to the two `Driver` classes. There is virtually no chance that the features would need to be recompiled.

So why did we choose the TEMPLATE METHOD? Because it was simpler. Because the data structure was not something that was going to change frequently. And because recompiling all the features cost just a few minutes.

So, even though the use of inheritance in the TEMPLATE METHOD pattern resulted in a more tightly coupled design, and even though the STRATEGY pattern conforms to the DIP better than the TEMPLATE METHOD pattern does, in the end *it wasn't worth the two extra classes* to implement the STRATEGY.

The Common Requirements of the Delivery Applications

The delivery programs had quite a bit of overlap. For example, the structure of the screen was the same in all vignettes. At the left of the screen was a window that contained nothing but a column of buttons. This window was called the "Command Window." The buttons in the Command Window acted as the controls for the application. They were labeled with terms such as, "Place Item," "Erase," "Move/Adjust," and "Zoom," and "Done." Clicking on these buttons drove the application through the desired behaviors.

5. Clearly, we didn't think of it in those terms. The names of the patterns hadn't been invented at the time we made this decision.

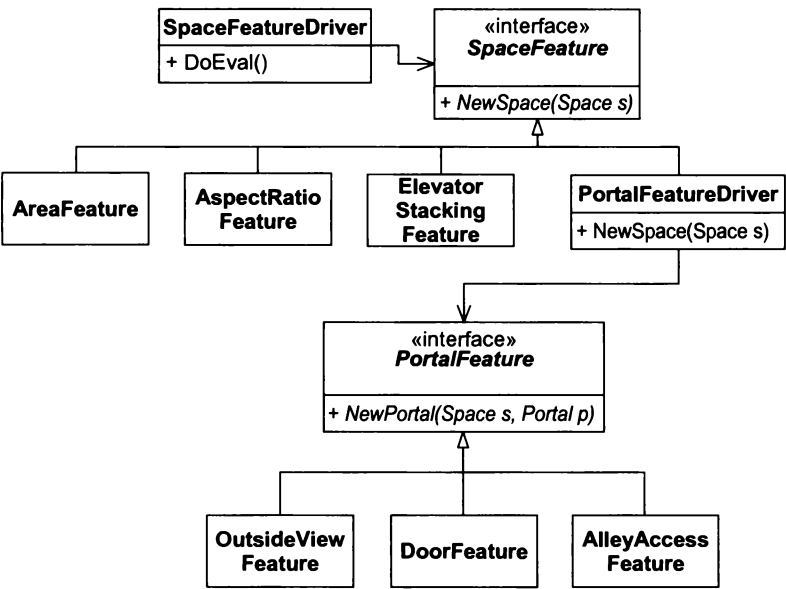


Figure 30-6 Floor-plan scoring structure with STRATEGY

To the right of the Command Window was the Task Window. This was a large scrollable and zoomable area into which the user could draw his solution. Commands initiated in the Command Window were typically used to modify the contents of the Task Window. Indeed, most of the commands that were initiated in the command window required significant interaction within the Task Window.

For example, in order to place a room on a floor plan, the user would click on the Place Item button in the Command Window. A menu of possible rooms would pop up. The user would select the kind of room he wished to place into the floor plan. Then the user would move the mouse into the Task Window and click at the location where he wanted the room to be placed. Depending upon the vignette, this might anchor the upper left corner of the room where the user clicked. The lower left corner of a stretchable room would then follow the motions of the mouse in the Task Window until the user clicked for the second time, anchoring the lower left corner at that position.

These actions were similar, though not identical, in each vignette. Some vignettes did not deal with rooms, but rather dealt with contour lines, property lines, or roofs. Though there were differences, the overall paradigm of operating within the vignettes was quite similar.

This similarity meant that we had a significant opportunity for reuse. We ought to be able to create an object-oriented framework that captured the bulk of the similarities and that allowed the differences to be expressed conveniently. In this we succeeded.

The Design of the Delivery Framework

The ETS framework finally grew to be nearly 75,000 lines of code. Clearly we cannot show all the details of that framework here. Thus, we have chosen two of the most illustrative elements of the framework to explore: the event model and the taskmaster architecture.

The Event Model. Every action taken by the user caused events to be generated. If the user clicked upon a button, an event named after that button was generated. If the user selected a menu item, an event named for that menu item was generated. Marshalling those events was a significant problem for the framework.

The reason for the problem was the fact that a very large portion of all the events could be handled by the framework, and yet each individual vignette might need to override the way the framework handled a particular

event. Thus, we needed to find a way to allow the vignettes the power to override the processing of events if needed.

The problem was complicated by the fact that the list of events was not closed. Each vignette could select its own particular set of buttons in the Command Window and its own particular set of menu items. Thus, the framework needed to marshal the events that were common to all vignettes, while allowing each vignette to override the default processing; and it needed to allow for the vignette to marshal its own vignette-specific events. This was not an easy task to accomplish.

As an example, consider Figure 30-7. This diagram⁶ shows a small portion of the finite state machine that would be used to marshal the events that occurred in the Command Window of a vignette. Each vignette had its own special version of this finite state machine.

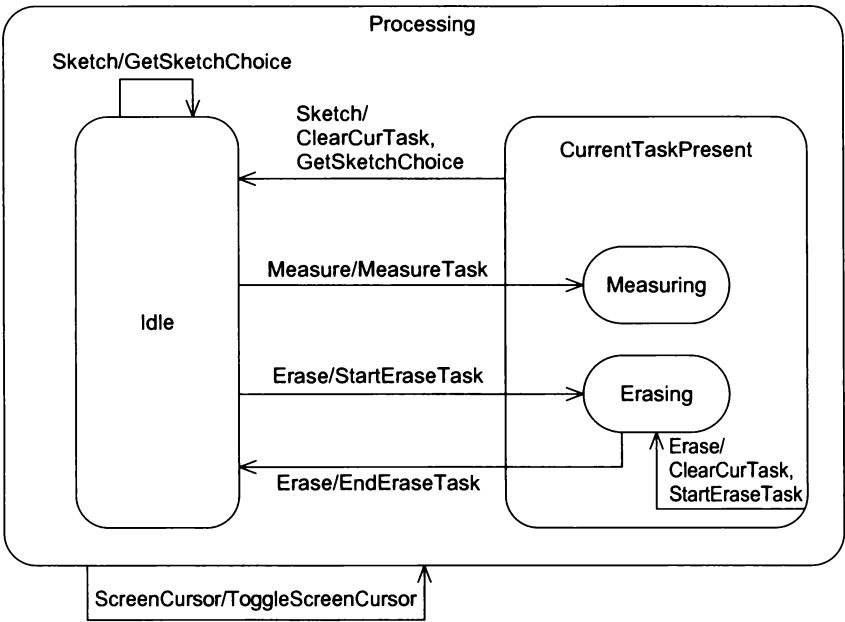


Figure 30-7 Command Window Event Processor

Figure 30-7 shows how three different kinds of events behaved. Let us first consider the simplest case, the `ScreenCursor` event. This event was generated when the user clicked on the Change Cursor button. Each time the user clicked this button, the cursor in the Task Window would toggle between an arrow and a full-screen cross hair. Thus, although the cursor changed state, no change of state occurred in the event processor.

When a user wants to delete an object he has drawn, he clicks on the Erase button. He then clicks on the item or items in the Task Window that he wants to delete. Finally, he clicks again on the Erase button to commit the deletion. The state machine in Figure 30-7 shows how the Command Window event processor deals with this. The first `Erase` event causes a transition from the `Idle` state to the `Erasing` state and starts up an `Erase Task`. We'll be talking more about `Tasks` in the next section. For now, it is sufficient that you know that the `Erase Task` will deal with any events that take place in the Task Window.

Notice that even in the `Erasing` state, the `ScreenCursor` event will still function properly and will not interfere with the erase operation. Notice also that there are two ways out of the `Erasing` state. If another `Erase` event occurs, then the `Erase Task` is ended, committing the erasure, and the state machine transitions back to the `idle` state. This is the normal way to end the erase operation.

6. The notation for state diagrams like this is fully described in Appendix B on page 489.

The other way to end the erase operation is to click on some of the other buttons in the Command Window. If, while erasure is in progress, you click on a Command Window button that starts up a different task (e.g., the Sketch button) then the Erase Task is aborted, and the deletion is cancelled.

Figure 30-7 shows how this cancellation works with the Sketch event, but there are many other events not shown that work the same way. If the user hits the Sketch button, either when the system is in the Erasing or Idle state, the system will transition to the Idle state and the GetSketchChoices function will be called. This function puts up the sketch menu, which contains a list of operations that the user can perform. One of those operations is measure.

When the user selects the measure item from the sketch menu, the Measure event occurs. This starts up the Measure Task. When measuring, the user may click on two points in the Task Window. The two points will be marked with tiny little cross hairs, and the distance between them will be reported in a small message window at the bottom of the screen. The user may then click on two more points, and two more, and two more, etc. There is no normal exit from the Measure Task. Instead, the user must click on a button that could start up another task, like Erase or Sketch.

The Event Model Design. Figure 30-8 shows the static model of the classes that implement the Command Window event processor. The hierarchy on the right represents the CommandWindow, while the hierarchy on the left represents the finite-state machine that translates events into actions.

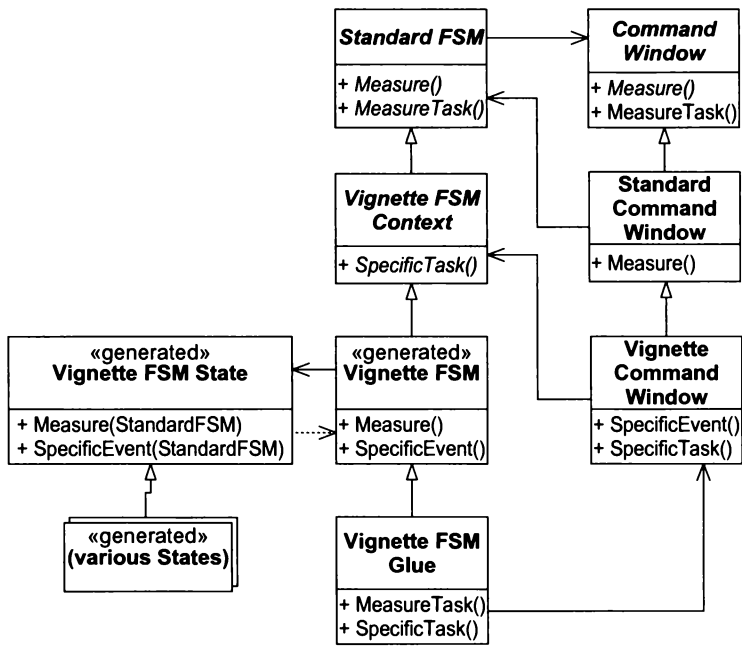


Figure 30-8 Command Window Event Processor Static Model

CommandWindow, StandardCommandWindow, and StandardFSM are framework classes. The rest are specific to the vignette. CommandWindow provides the implementations for standard actions, such as MeasureTask and EraseTask.

Events are received by the VignetteCommandWindow. They are passed to the finite-state machine, which translates them into actions. The actions are then passed back to the CommandWindow hierarchy, which implements them.

The CommandWindow class provides implementations for standard actions such as MeasureTask and EraseTask. A “standard action” is an action that is common to all vignettes. StandardCommandWindow

provides for the marshalling of standard events to the finite-state machine. `VignetteCommandWindow` is specific to the vignette and provides for both the implementation of specific actions and the marshalling of specific events. It also allows the standard implementations and marshalling to be overridden.

Thus, the framework provides default implementations and marshalling for the all the common tasks. But any one of those implementations or marshallings can be overridden by the vignette.

Tracing a Standard Event. Figure 30-9 shows how a standard event gets marshalled to the finite state machine and translated to a standard action. Message 1 is the `Measure` event. It comes from the GUI and is passed to the `VignetteCommandWindow`. The default marshalling for this event is provided by the `StandardCommandWindow`, so it forwards the event to the `StandardFSM` in message 1.1.

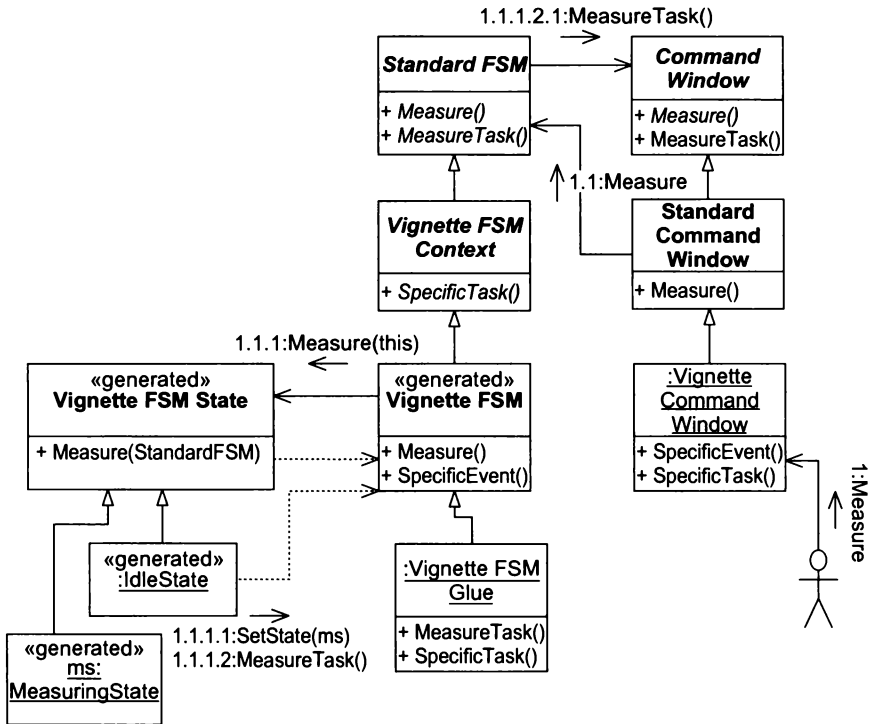


Figure 30-9 Processing the Measure Event

`StandardFSM` is a framework class that provides an interface for all the incoming standard events and all the outgoing standard actions. None of these functions are implemented at this level. `VignetteFSMContext` adds interfaces, but no implementations, for vignette-specific events and actions.

The real work of translating an event to an action takes place in the `VignetteFSM` and `VignetteFSMState` classes. `VignetteFSM` contains the implementations for all the event functions. So, the `1.1:Measure` message gets deployed down to this level. `VignetteFSM` responds by sending `1.1.1.1:Measure(this)` to the `VignetteFSMState` object.

`VignetteFSMState` is an abstract class. There are derivatives of this class for each state of the finite-state machine. In Figure 30-9, we presume that the current state of the FSM is `Idle`. (See Figure 30-7.) Thus the `1.1.1.1:Measure(this)` message gets deployed to the `IdleState` object. This object responds by sending two messages back to `VignetteFSM`. The first is `1.1.1.1.1:SetState(ms)`, which changes the state of the FSM to the `Measuring` state. The second message is `1.1.1.1.2:MeasureTask()`, which is the action required by the `Measure` event in the `Idle` state.

The MeasureTask message is finally implemented in the VignetteFSMGlue class, which recognizes the action as a standard action declared in CommandWindow, and therefore routes it there in message 1.1.1.2.1:MeasureTask, thus completing the circuit.

The mechanism employed for converting events to actions is the STATE pattern. We make prodigious use of this pattern in many areas of the framework, as the following sections will show. The «generated» stereotype that appears in the STATE pattern classes indicates that these classes are automatically generated by SMC.

Tracing a Vignette Specific Event. Figure 30-10 shows what happens when a vignette-specific event occurs. Once again, message 1:SpecificEvent is caught by the VignetteCommandWindow. However, since the marshalling for specific events are implemented at this level, it is the VignetteCommandWindow that sends message 1.1:SpecificEvent. Moreover, it sends it to the VignetteFSMContext class where the SpecificEvent method was first declared.

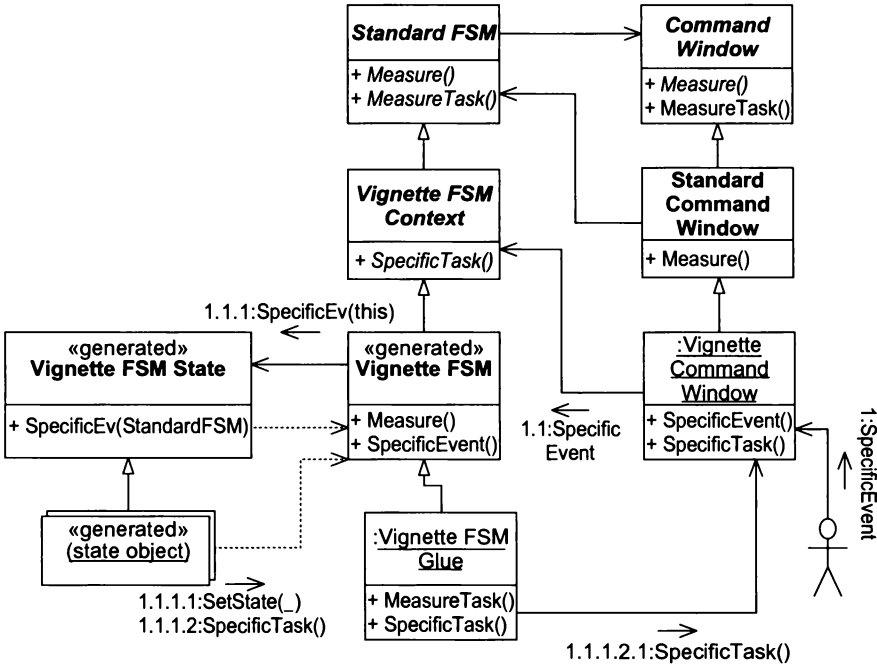


Figure 30-10 Processing a specific event

Once again, the event is deployed to VignetteFSM, which, in message 1.1.1:SpecificEv, negotiates with the current state object to generate the corresponding action. As before, the state object replies with two messages, 1.1.1.1:SetState and 1.1.1.2:SpecificTask.

Again, the message is deployed down to VignetteFSMGlue. However, this time, it is recognized as a vignette specific action and is therefore routed directly to the VignetteCommandWindow where specific actions are implemented.

Generating and Reusing the Command Window State Machine. At this point, you might be wondering why it requires so many classes to marshal the events and actions in this fashion. Consider, however, that, although there are many classes, there are very few objects. Indeed, the instantiated objects are simply VignetteCommandWindow, VignetteFSMGlue, and the various state objects, which are trivial and automatically generated.

Though the flow of messages seems complex, it is really quite simple. The window detects an event, passes it to the FSM for translation to an action, and accepts the action back from the FSM. The rest of the complexity has to do with separating standard actions known to the framework from specific actions known to the vignette.

Another factor impinging on our decision to partition the classes as we did was our use of SMC to automatically generate the finite-state machine classes. Consider the following description, and refer back to Figure 30-7:

```
Idle
{
    Measure Measuring MeasureTask
    Erase Erasing StartEraseTask
    Sketch Idle GetSketchChoice
}
```

Notice that this simple text describes all of the transitions that occur while the state machine is in the `Idle` state. The three lines in the braces identify the event that triggers the transition, the target state of the transition, and the action performed by the transition.

SMC⁷ accepts text in this form and generates the classes denoted with «generated». The code generated by SMC requires neither editing nor any inspection.

Using SMC to generate the state machine makes the creation of this part of the event processor for a vignette quite simple. The developer must write the `VignetteCommandWindow` with its corresponding implementations for specific events and actions. The developer must also write the `VignetteFSMContext`, which simply declares interfaces for the specific events and actions. And then the developer needs to write the `VignetteFSMGlue` class, which simply dispatches actions back to `VignetteCommandWindow`. None of these tasks are particularly challenging.

There is one other thing the developer must do. He must write the description of the finite-state machine for SMC. This state machine is actually rather complicated. The diagram in Figure 30-7 doesn't do it justice at all. A real vignette must deal with many dozens of different events, each of which can have remarkably different behaviors.

Fortunately, most of the vignettes behave in roughly the same way. So we were able to use a standard state-machine description as a model and make relatively minor modifications to it for each vignette. Thus, each vignette had its own FSM description.

This approach is somewhat unsatisfying since the FSM description files were very similar. Indeed, there were several occasions when we were forced to change the generic state machine, which meant that we had to make identical or nearly identical changes in each of the FSM description files. This is both tedious and error prone.

We might have invented yet another scheme for separating the generic portion of the FSM description from the specific portion, but in the end we didn't feel it was worth the effort. This is a decision we have chided ourselves for more than once.

The Taskmaster Architecture

We have seen how events get converted into actions and how that conversion is dependent on a relatively complex finite-state machine. Now we are going to look at how the actions themselves are processed. It should come as no surprise that the core of each action is also driven by a finite state machine.

Let's consider the `MeasureTask` that we discussed in the previous section. The user invokes this task when he wants to measure the distance between two points. Once invoked, the user clicks on a point in the `TaskWindow`. A small cross hair will appear at that point. As the user then moves the mouse around, a stretchable line is drawn

7. SMC, the State Machine Compiler, is freeware available from <http://www.objectmentor.com>.

from the click point to the current mouse position. Moreover, the current length of that line is displayed in a separate message window. When the user clicks for the second time, another cross hair is drawn, the stretchable line disappears, and the final distance between the two points is displayed in the message box. If the user then clicks again, the process starts over.

Once the event processor has selected the `MeasureTask`, as shown in Figure 30-9, the `CommandWindow` creates the actual `MeasureTask` object, which then runs the finite-state machine shown in Figure 30-11.

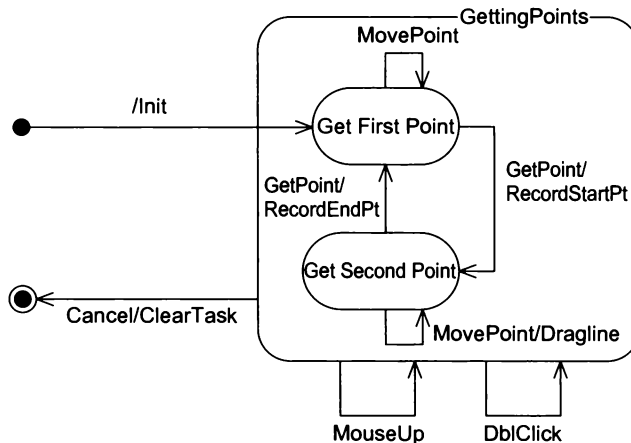


Figure 30-11 Finite-State Machine for the Measure Task.

The `MeasureTask` begins its life by invoking its `init` function and then occupying the `GetFirstPoint` state. GUI events that occur in the `TaskWindow` are routed to the task that is currently executing. Thus, when the user moves the mouse in the `TaskWindow`, the current task receives a `MovePoint` message. Notice that, in the `GetFirstPoint` state, this does nothing (as expected).

When the user finally clicks in the `TaskWindow`, a `GotPoint` event occurs. This causes a transition to the `GetSecondPoint` state and invokes the `RecordStartPt` action. This action will draw the first cross hair and will also remember the location that the mouse was clicked as the starting point.

In the `GetSecondPoint` state, a `MovePoint` event causes the `Dragline` action to be invoked. This action sets XOR mode⁸ and draws a line from the remembered starting point to the current mouse location. It also computes the distance between these two points and displays it in the message window.

`MovePoint` events occur frequently whenever the mouse is being moved over the `TaskWindow`, so the motion of the line, and the length displayed in the message window, will appear to continuously update as long as the mouse is in motion.

When the user clicks for the second time, we transition back to the `GetFirstPoint` state and invoke the `RecordEndPt` action. This action turns off XOR mode, erases the line between the first point and the current mouse position, draws a cross hair at the click point, and displays the distance between the starting point and the click point in the message window.

This sequence of events repeats as long as the user desires. It only terminates when the task is cancelled by the `CommandWindow`, probably in response to the user clicking on a command button.

Figure 30-12 shows a somewhat more complex task—that of drawing a “two-point” box. A two-point box is a rectangle that is drawn on the screen with two clicks. The first click anchors one corner of the box. A stretchable box then follows the mouse. When the user clicks for the second time, the box is made permanent.

8. XOR mode is a mode in which the GUI can be placed. It vastly simplifies the problem of dragging stretchable lines or shapes over existing shapes on the screen. If you don’t understand this, don’t worry about it.

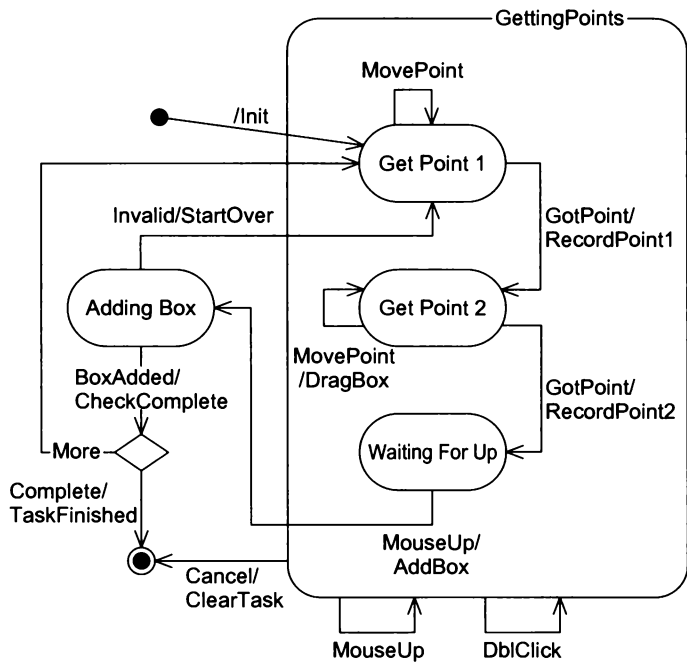


Figure 30-12 Two-Point Box

As before, the task begins in the `GetPoint1` state, after invoking `Init`. In this state, the motion of the mouse is ignored. When the mouse is clicked, we transition to `GetPoint2` and invoke the `RecordPoint1` action. This action records the click point as the starting point.

In the `GetPoint2` state, mouse motion causes the `DragBox` action to be invoked. This function sets `XOR` mode and draws the stretchable box from the starting point to the current mouse position.

When the mouse goes down for the second time, we transition to the `WaitingForUp` state and invoke `RecordPoint2`. This function simply records the final point for the box. It does not cancel `XOR` mode, erase the stretchable box, or draw the real box, because we aren't sure if the box is valid yet.

At this point, the mouse is still down, and the user is about to lift his finger off the mouse button. We need to wait for this to occur, otherwise some other task might get the mouse-up event and confuse things. While we are waiting, we ignore any motion of the mouse, leaving the box anchored at the last click point.

Once the mouse comes up, we transition to the `AddingBox` state and invoke the `AddBox` function. This function checks to see if the box is valid. There are many reasons why a box might be invalid. It might be degenerate (i.e., its first and final points are the same), or it might conflict with something else on the drawing. Each vignette has the right to reject something that the user has attempted to draw.

If the box is found to be invalid, an `Invalid` event is generated, and the state machine transitions back to `GetPoint1` while invoking the `StartOver` function. If, however, the box was found to be valid, then the `BoxAdded` event is generated. This causes the `CheckComplete` function to be called. This is another vignette-specific function. It determines whether the user should be allowed to continue drawing more boxes or whether the task should be completed.

There are literally dozens of such tasks in the framework. Each is represented by a derivative of the `Task` class. (See Figure 30-13.) Each task has a finite-state machine within it, and they are substantially more complex than we have been able to show here. Again, each of those state machines was generated by SMC.

Figure 30-13 shows the Taskmaster architecture. This architecture connects the `CommandWindow` to the `TaskWindow`, and creates and manages the tasks that the user has selected.

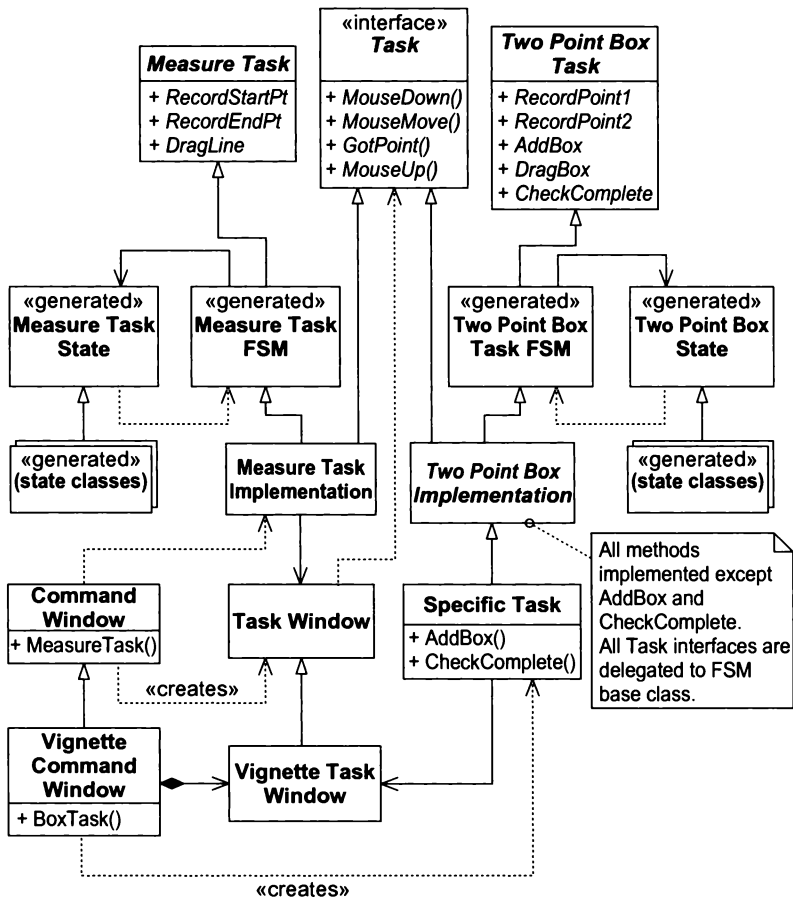


Figure 30-13 Taskmaster Architecture

The diagram shows the two tasks whose FSMs were depicted in Figures 30-11 and 30-12. Notice the use of the STATE pattern and the classes generated in each of the tasks. All of these classes, down to **MeasureTaskImplementation** and **TwoPointBoxImplementation** are part of the framework. Indeed, the only classes that the developer must write are **VignetteTaskWindow** and the specific derivatives of the task classes.

The classes **MeasureTaskImplementation** and **TwoPointBoxImplementation** represent the many different tasks that are contained in the framework. But note that these classes are abstract. There are a few functions, such as **AddBox** and **CheckComplete**, that are not implemented. Each vignette must implement these functions as needed for their own concerns.

Thus, the tasks contained by the framework govern the bulk of the interactions in all the vignettes. Whenever a developer needs to draw a box, or an object related to a box, that developer can derive a new task from **TwoPointBoxImplementation**. Or whenever he needs to simply place some object on the screen with a single click, he can override **SinglePointPlacementTask**. Or if he needs to draw something based upon a polyline, he can override **PolylineTask**. And these tasks manage the interactions, perform any needed dragging, and give the engineer the hooks needed to validate and create the objects he needs.

Conclusion

Of course there is much more that we could have talked about in this chapter. We could have discussed the elements of the framework that dealt with computational geometry or the bits that dealt with storing and reading the

solutions to and from files. We could have discussed the structure of the parameter files that allowed each vignette application to drive many different variations of the same vignette. Unfortunately, neither space nor time permits.

However, we think we covered the aspects of the framework that were most instructional. The strategies we employed in this framework can be employed by others to make their own reusable frameworks.

Bibliography

1. Booch, Grady. *Object-Oriented Design with Applications*. Redwood City, CA: Benjamin Cummings, 1991.
2. Booch, Grady. *Object Solutions*. Menlo Park, CA: Addison–Wesley, 1996.
3. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

APPENDIX A

UML Notation I: The CGI Example

The analysis and design of software comprise a process that cries out for some kind of notation. There have been many attempts at creating such a notation. Flowcharts, data-flow diagrams, entity-relationship diagrams, etc.

The advent of object-oriented programming saw an explosion in notation. There were literally dozens of competing notations for the representation of object-oriented analyses and designs.

The most popular of these notations were the following:

- Booch 94.¹
- OMT (Object Modeling Technique) as described by Rumbaugh, et al.²
- RDD (Responsibility Driven Design) as described by Wirfs-Brock, et al.³
- Coad/Yourdon as described by Peter Coad and Ed Yourdon⁴

Of these, Booch 94 and OMT were by far the most important. Booch 94 was hailed as a strong design notation, whereas OMT was considered to be stronger as an analysis notation.

This dichotomy is interesting. In the late 1980s and early 1990s, it was considered one of the advantages of object orientation that analysis and design could be represented by the same notation. Possibly, this was a reaction to the strong separation between the notations for structured analysis and structured design. Crossing the chasm from structured analysis to structured design was well known to be difficult.

When object-oriented notations first exploded upon the scene, it was felt that the same notation would serve both analysis and design. Yet as the decade progressed, analysts and designers began to migrate to their favorite notations. Analysts tended to favor OMT, and designers tended to favor Booch 94. Thus, it appeared that one notation would not really suffice. A notation tuned for analysis was not appropriate for design, and vice versa.

The UML is a single notation, but it has broad applicability. Parts of the notation are usable for analysis; other parts are usable for design. Thus, both analysts and designers can use the UML.

In this chapter, we will be presenting the UML notation from both views. First we will describe an analysis, and then we will go on to describe a design. This description will take place in the form of a miniature case study.

1. [BOOCH94].
2. [RUMBAUGH91].
3. [WIRFS90].
4. [COAD91A].

Please note that this ordering of analysis first, and then design, is artificial, and is not intended as a recommendation. Indeed, none of the other case studies in this book makes the distinction between the two. I have presented it this way here simply to illustrate how UML can be used at different levels of abstraction. In a real project all levels of abstraction are produced concurrently—not in sequence.

Course Enrollment System: Problem Description

Suppose that we are working for a company that offers professional training courses in object-oriented analysis and design. The company needs a system that keeps track of the courses being taught and the students that are enrolled. See the “Course Enrollment System” sidebar.

Course Enrollment System

Users must be able to view a menu of the available course offerings and select the course in which they wish to enroll. Once selected, a form should pop up that allows the user to enter the following information:

- Name
- Phone number
- Fax number
- e-mail address

There should be a way for the user to select the manner in which he wants to pay for the course. Those methods may be one of the following:

- Check
- Purchase order
- Credit card

If the user wishes to pay by check, then the form should prompt him for the check number.

If the user wishes to pay by credit card, then the form should prompt him for the credit card number, expiration date, and the name as it appears on the card.

If the user wishes to pay by purchase order, then the form should prompt him for the purchase order number (PO#), the name of the company, and the name and phone number of someone in the accounts payable department.

Once all this information has been filled out, the user will click on a ‘submit’ button. Another screen will pop up, which summarizes all the information that the user entered. It will instruct the user to print the screen, sign the printed copy, and fax it to the number of the enrollment center.

It should also e-mail an enrollment summary to our enrollments clerk and to the user.

The system will know the maximum number of students for each class, and will automatically mark the class offering as “Sold Out” once that limit has been hit.

The enrollments clerk will be able to e-mail messages to all students enrolled in a given course by bringing up a special form and selecting the course. This form will allow the clerk to type a message and then hit a button that will send it to all students currently enrolled in the selected course.

(continues on next page)

The enrollments clerk will also be able to bring up a form that shows the status of all students for classes that have already been taught. The status will indicate whether or not the student was in attendance and whether or not the student's payment has been received. This form can be brought up on a course-by-course basis. Or the enrollments clerk can ask to see a list of all students who have outstanding balances.

Identifying Actors and Use Cases. One of the tasks of requirements analysis is to identify the actors and use cases. It should be noted that in a real system, these are not necessarily the first tasks that are appropriate. But for the purposes of this chapter, they are where I choose to begin. In reality, the place you begin is less important than the act of beginning.

Actors

Actors are entities that interact with the system, but are outside the system. Often they are roles played by the users of the system. Sometimes, however, they can be other systems. In this example, all the actors correspond to human users.

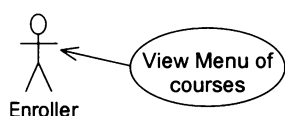
Enroller. This actor enrolls a student in a course. It interacts with the system to select the appropriate course and enter the information about the student and the method of payment.

Enrollment Clerk. This actor receives e-mail notification of each enrollment. It also sends e-mail notices to students and receives reports about enrollments and payments

Student. This actor receives e-mail confirmation of enrollment and email notifications from the Enrollment Clerk. The Student attends the courses it is enrolled in.

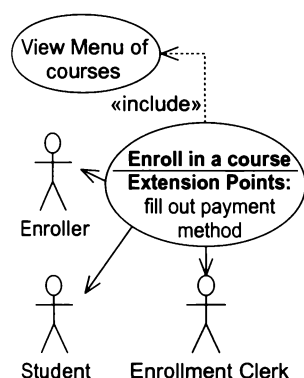
Use Cases

Having determined the actors, we specify the interactions of these actors with the system. These specifications are called "use cases." A use case describes the interaction between an actor and the system from the actor's point of view. None of the inner workings of the system are discussed, nor is the user interface described in any detail.



Use Case #1: View Menu of Courses. The Enroller requests a list of courses that are currently available in the course catalog. The system displays the list of courses. Included in that list are the name, time, place, and cost of the course. The list will also show the number of students allowed in the course and whether or not the course is currently sold out.

Use Case Notation. The above diagram shows an actor and a use case within a use-case diagram. The actor is the little stick figure, and the use-case is the ellipse. The two are bound together by an association that shows the direction of the data flow.



Use Case #2: Enroll in a course. The Enroller first views the menu of courses (Use Case #1). The Enroller selects a course for enrollment from the menu. The system prompts the Enroller for the name, phone number, fax number, and email address of the student. The system also prompts the Enroller for the preferred payment method.

Extension point: fill out payment method

The Enroller submits the enrollment form. The Student and the Enrollment Clerk are sent email confirming the enrollment. The Enroller is shown a confirmation of the enrollment and is asked to print that confirmation, sign it, and fax it to a specific fax number.

Extending and Using Use Cases. Use Case #2 has an extension point. This means that other use cases will be extending this use case. The extending use cases are presented below as #2.1, #2.2, and #2.3. Their descriptions are inserted into the previous use case at the extension point. They describe the optional data that need to be entered depending on the selected payment method.

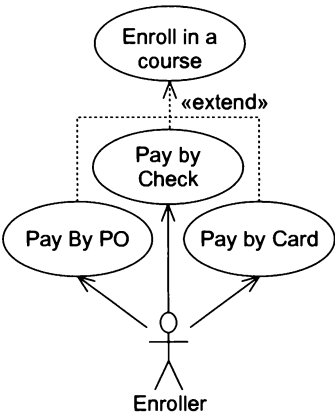
Use Case #2 also has a «include» relationship with Use Case #1, “View Menu of Courses.” This means that the description of Use Case #1 is inserted into the appropriate location of Use Case #2.

Note the difference between extending and including. When one use case includes another, the including use case references the included use case. However, when one use case extends another, neither use case refers to the other. Instead, an extending use case is selected based upon context and is inserted into the extended use case at the appropriate point.

We use the «include» relationship when we want to make the structure of the use cases more efficient by collapsing repeated operations into smaller use cases that can be shared among many other use cases. The goal is to manage change and eliminate redundancy. By moving the common parts of many use cases into a single included use case, when the requirements of that common part change, only the single included use case must change.

We use the «extend» relationship when we know that there are many alternatives or options within a use case. We separate the invariant part of the use case from the variable parts. The invariant part becomes the use case that is extended, and the variable parts become the extending use cases. The goal, once again, is change management. In the current example, if new payment options are added, new extending use cases will have to be created, but no existing use cases need to be modified.

Notation for the «include» Relationship. The diagram for Use Case #2 shows that the “Enroll in a Course” use case is bound to the “View Menu of Courses” use case by a dashed line that terminates in an open arrowhead. The arrowhead points at the included use case and has the stereotype «include».

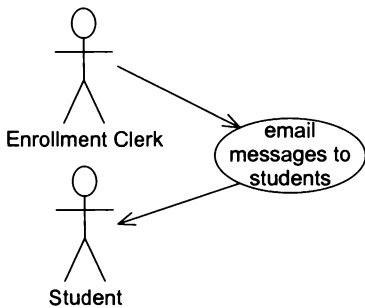


Use Case #2.1: Pay by Purchase Order. The Enroller is prompted for the PO#, the name of the company, and the name and phone number of someone in the accounts-payable department.

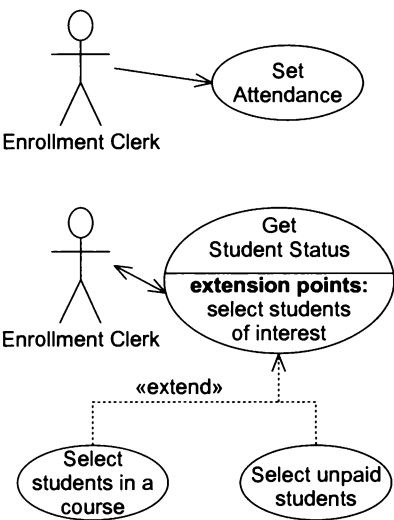
Use Case #2.2: Pay by Check. The Enroller is prompted for the check number.

Use Case #2.3: Pay by Card. The Enroller is prompted for the credit card number, expiration date, and the name as it appears on the card.

Notation for Use-Case Extension. The extending use cases shown above are connected to the extended use case through «extend» relationships. This relationship is drawn as a line that connects the two use cases. Once again, dashed, the line has an open arrowhead. The arrowhead points at the extended use case and has the stereotype «extend».



Use Case #3: Email Messages to Students. The Enrollment Clerk selects a course and enters the text of a message. The system sends the message to the e-mail addresses of all the students currently enrolled in that class.



Use Case #4: Set Attendance. The Enrollment Clerk selects a class and a student that is currently enrolled in that class. The system presents the student and shows whether that student was in attendance and whether or not that student’s payment has been received. The Enrollment Clerk may then change the attendance or payment status.

Use Case #5: Get Student Status. The Enrollment Clerk selects the students of interest.

Extension Point: ‘select students of interest’
The system presents the attendance and payment status of the selected students in a single report.

Extensions There are two use cases that extend Get Student Status.

Use Case #5.1: Select Students in a Course. The system shows a list of all the courses. The Enrollment Clerk selects a course. The system selects all students in that course.

Use Case #5.2: Select Unpaid Students. The Enrollment Clerk indicates that the system should select all unpaid students. The system selects all students who have been marked in attendance and whose payment status indicates that payment has not been received.

Use-Case Reprise. The use cases we have created here describe how the users expect the system to behave. Notice that they do not talk about the details of the user interface. They don’t mention icons, menu items, buttons, or scrolling lists. Indeed, even the original specification said more about the user interface than the use cases do. This is intentional. We want the use cases to be lightweight and easy to maintain. As written, these use cases are valid for an extremely large set of possible implementations.

System-Boundary Diagram. The entire set of use cases can be shown in summary by the system-boundary-diagram in Figure A-1. This diagram shows all the use cases in the system surrounded by a rectangle that represents the system boundary. The actors are set outside the system and are connected to the use cases with associations that show the direction that data flows.

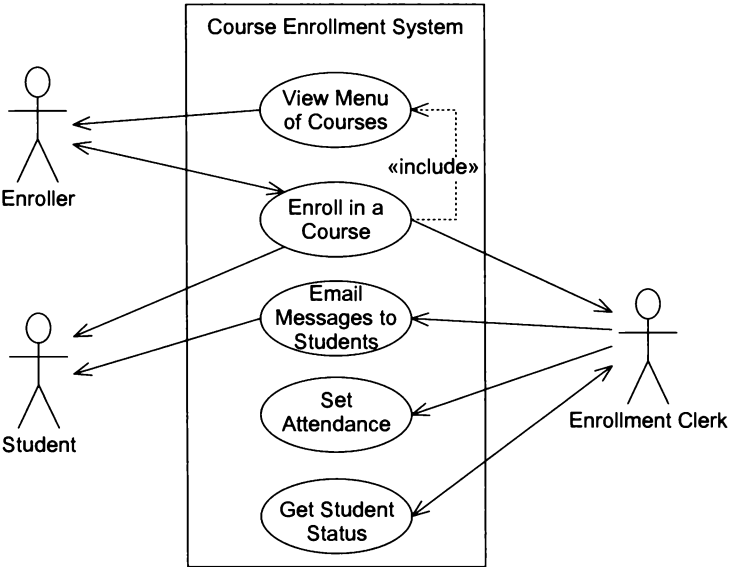


Figure A-1 System-Boundary Diagram

What Do We Use these Diagrams for? Use-case diagrams, including system-boundary-diagrams, are not software-structure diagrams. They do not give us any information at all about the partitioning of the software elements of the system to be created. These diagrams are used for human communication, primarily between the analysts and the stakeholders. They help to organize the functionality of the system in terms of the different kinds of system users.

Moreover, these diagrams can be quite helpful when presenting the system description to the various kinds of users. Each different kind of user is interested chiefly in his own use cases. The connection of the actors to the use cases serves to focus each different kind of user on the use cases that he or she will be using. In very large systems, you may wish to organize the system boundary diagrams according to actor type so that all different user types can look at the subset of use cases that concerns them.

Finally, remember Martin's first law of documentation: *Produce no document unless its need is immediate and significant.*

These diagrams can be useful, but they are often unnecessary. You should not think of them as required or essential. If you have a need for them, then draw them. Otherwise, wait until you have a need.

The Domain Model

The domain model is a set of diagrams that helps to define the terms that appear in the use cases. These diagrams show the key objects within the problem and their interrelationships. Much pain and suffering have resulted from the tendency to assume that this model is also a model of the software to be built. It is important for both the analyst and the designer to realize that a domain model is a descriptive tool that is used to help humans record their decisions and communicate with each other. The objects in the domain model do not necessarily correspond to the object-oriented design of the software, nor would such a correspondence be a strong benefit.⁵

In Booch 94 and OMT, domain-model diagrams were indistinguishable from diagrams that represented software structure and design. In the worst cases, the domain model diagrams were taken to be high-level design documents and were therefore used to set the high-level structure of the software itself.

To help avoid this kind of mistake, we can take advantage of the features of UML and use a special kind of entity called a «type» in domain models. A «type» represents a role that an object can play. A «type» can have operations and attributes as well as associations with other «type» entities. However, a «type» does not represent a class or object in the design sense. It does not represent an element of software, and it does not map directly to code. It represents a conceptual entity used in the description of the problem.

The Course Catalog. The first domain abstraction we will consider is the course catalog. This abstraction represents the list of all the courses that are offered. We show this in the domain model (see Figure A-2) for the Course Catalog entity by depicting two entities that represent abstractions in the domain: the CourseCatalog entity and the Course entity. The CourseCatalog entity offers many Course entities.

Domain-Model Notation. The notation used in Figure A-2 depicts the two domain abstractions as UML classes with the «type» stereotype. (See sidebar, “Overview of UML Class Notation and Semantics.”) This indicates that the classes are conceptual elements of the problem domain and not directly related to software

5. [JACOBSON], p. 133, “We do not believe that the best (most stable) systems are built by *only* using objects that correspond to real-life entities...”

And also on p. 167, “In [other] methods, this [domain] model will also form a base for the actual implementation; that is, the objects are directly mapped onto classes during implementation. However, this is not the case in OOSE, [...]. Our experience with such an approach tells us differently. Instead we develop an analysis model that is more robust and maintainable in the face of future changes rather than using a problem domain model to serve as the base for design and implementation.

[BOOCH96], p. 108, “...there is a tendency in immature projects to consider the domain model resulting from analysis as ready to code [...], thus skipping any further design. Healthy projects recognize that there is still quite some work to do, involving issues such as concurrency, serialization, safety, distributions, and so on, and that the design model may end up looking quite different in a number of subtle ways.

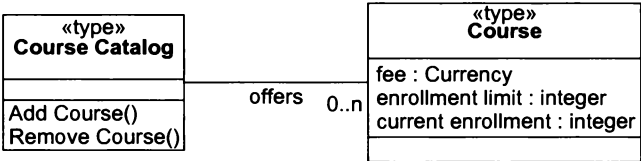


Figure A-2 Domain model for the Course Catalog entity

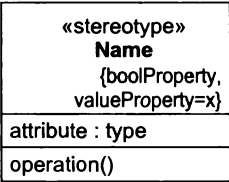
classes. Notice that the `CourseCatalog` entity has two operations: `AddCourse` and `RemoveCourse`. In a `«type»`, operations correspond to *responsibilities*. Thus, the `CourseCatalog` has the responsibility to be able to add and remove courses. Again, these are concepts, not the specifications of member functions within real classes. We use them to communicate with users, not to specify a software structure.

By the same token, the attributes shown in the `Course` entity are concepts. They indicate that the `Course` entity should be responsible to remember its fee, enrollment limit, and current enrollment.

Overview of UML Class Notation and Semantics

In UML, a class is drawn as a rectangle with three compartments. The first compartment specifies the name of the class. The second specifies its attributes and the third its operations.

Within the name compartment, the name can be modified by a stereotype and by properties. The stereotype appears above the name and is enclosed within guillemets (French quotation marks `«»`). Properties appear below and to the right of the name and are enclosed within braces. (See the following diagram.)



Stereotypes are names that refer to the “kind” of UML class being represented. In UML, a class is simply a named entity that can have attributes and operations. The default stereotype is `«implementation class»` in which case the UML class corresponds directly to the software notion of a class in a language like C++, Java, Smalltalk, or Eiffel. The attributes correspond to member variables, and the operations correspond to member functions.

However, with the stereotype of `«type»`, a UML class does not correspond to a software entity at all. Rather it corresponds to a conceptual entity that exists in the problem domain. The attributes represent the information that logically belongs to that conceptual entity, and the operations represent the responsibilities of that conceptual entity.

There are several other predefined stereotypes that we will be discussing later in this chapter. You are free to create your own stereotypes, too. However, a stereotype is more than just a comment. It specifies the way that all elements of a UML class should be interpreted. So if you make a new stereotype, define it well.

(continues on next page)

Properties are primarily structured comments. Properties are specified in a comma-separated list between braces. Each property is a name = value pair separated by an equal sign (=). If the equal sign is omitted, then the property is assumed boolean and given the value of “true.” Otherwise the type of the value is a string.

There are several predefined properties that we will be discussing later in this chapter. However, you are free to add your own at any time. For example, you might create properties like this: {author=Robert C. Martin, date=12/6/97, SPR=4033}

Concept vs. Implementation and the Use of Cloud Icons. I have been placing a lot of stress upon the difference between a class at the conceptual level (i.e., a «type») and a class at the design or implementation level. I feel that this is appropriate because of the danger that conceptual diagrams might be mistaken as specifications for the structure and architecture of the software. Remember, conceptual diagrams are to help communications with the *stakeholders* and are therefore devoid of all the technical issues of software structure.

The use of stereotypes to distinguish between these kinds of diagrams could be overlooked. UML classes for domain models look very much like UML classes for design and implementation. Fortunately, UML allows us to substitute different icons for different stereotypes. Therefore, to further enhance the difference between these kinds of diagrams, we will use a cloud icon to represent «type» classes from now on. This changes the domain model for the Course Catalog entity in Figure A-2 to look like Figure A-3.

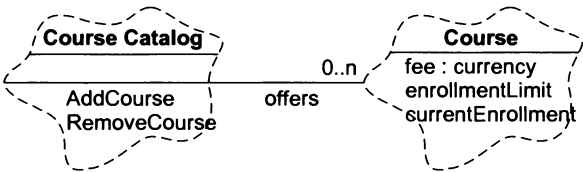


Figure A-3 Domain Model for Course Catalog using Cloud Icons

Completing the Domain Model. So far, the domain model shows a course catalog that contains all the offered courses. But there is a problem here. What do we mean by a course? The same course can be offered at many different times and locations, and it can be taught by many different instructors. Clearly we need two different entities. Let’s call the first a *Course*. It represents the course itself, but not the dates, locations, or instructors. Let’s call the second a *Session*. It represents the date, location, or instructor for a specific course. (See Figure A-4.)

Notation. The lines that connect the entities are called associations. All the associations in Figure A-4 are named, although this is not a rule. Notice that the names are verbs or verb phrases. The little black triangle next to the name points at the predicate of the sentence formed by the two entities and the association. Thus, “Course Catalog offers many Courses,” “Session Schedule schedules many Sessions,” and “Many students are enrolled in a Session.”

I used the word “many” in the previous sentences wherever the “0 . . *” icon was present on the corresponding relationship. This icon is one of several different *multiplicity* icons that can be placed on the end of an association. They indicate the number of entities that participate in the association. The default multiplicity is “1.” (See the sidebar entitled: “Multiplicities.”)

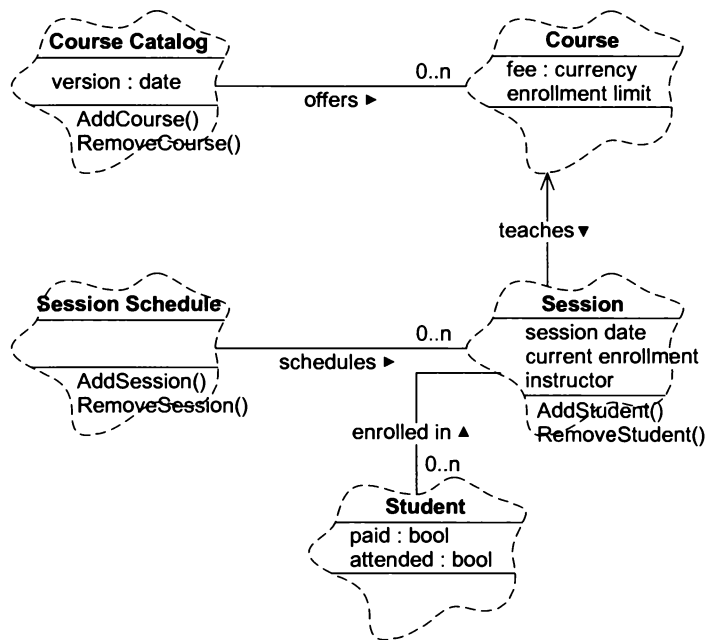


Figure A-4 Courses and Sessions

Multiplicities

There are several multiplicity icons that can be used to adorn associations. They include the following:

- 0..* zero to many
- * zero to many
- 1..* one to many
- 0..1 zero or one
- 6 exactly six
- 3..6 between three and six
- 3,5 three or five

Any nonnegative integers can be used between dots or separated by commas.

Associations are assumed to be bidirectional unless there is an arrowhead present. A bidirectional association allows the two entities to know about each other. For example, it is clear that the CourseCatalog entity ought to know about its Courses, and it seems reasonable that each Course entity could know about the CourseCatalog it is listed in. The same is true for the SessionSchedule and the Session.

The presence of an arrowhead restricts the knowledge to the direction indicated. Thus, Sessions know about Courses, but Courses do not know anything about Sessions.

Use Case Iterations. This diagram immediately tells us two things. First, the use cases employ the wrong language in a number of places. Where they talk about course catalogs and courses, they should be talking about session schedules and sessions. Secondly, there are quite a few use cases that were left out. The CourseCatalog and the SessionSchedule need to be maintained. Courses need to be added and removed from the CourseCatalog, and Sessions need to be added and removed from the SessionSchedule.

Thus, by creating a domain model, we better understand the problem at hand. That better understanding helps us to improve and augment the use cases. This iteration between the two is natural and necessary.

If we were pursuing this case study to its conclusion, we would show the changes implied above. But in the interest of efficiently presenting the notation, we will skip the iteration of the use cases.

The Architecture

Now we get down to the business of designing software. The architecture represents the software structures that form the skeleton of the application. The classes and relationships of the architecture map very closely to code.

Deciding the Software Platform. Before we can begin, however, we must understand the software platform in which this application is going to run. We have a number of choices.

- 1. A Web-based CGI application. The enrollment and other forms would be accessed by a Web browser. The data would reside at the Web server, and CGI scripts would be invoked by the Web browser to access and manipulate the data.
- 2. A database application. We could purchase a relational database and use the forms package and 4GL to write the application.
- 3. Visual XXX. We could purchase a visual programming language. The human interface could be created using the visual construction tools. These tools would invoke the software functions needed to store, retrieve, and manipulate the data.

There are, of course, other options. We could write the whole thing in C without any library or tool support other than the compiler. But this would be silly. The tools are there, and they work. We should use them.

For the purposes of our example, we will assume a Web-based application. This makes sense since then enrollers can be located anywhere in the world; and the enrollment service can be offered on the Internet.

Web Architecture. We need to decide the overall architecture of the Web applications. How many Web pages will there be, and what CGI programs will they invoke? Figure A-5 shows how we might begin to specify this.

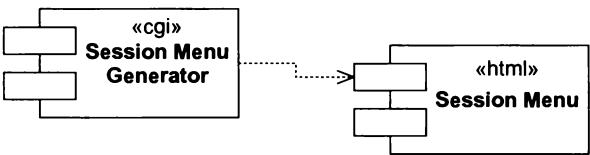


Figure A-5 Session Menu Architecture

Notation. Figure A-5 is a component diagram. The icons depict physical software components. We have used the stereotype to specify the kind of components. The diagram shows that the Session Menu is displayed as an HTML Web page that is generated by a CGI program called the Session Menu Generator. The dashed arrow between the two components is a dependency relationship. Dependency relationships denote which components have knowledge of other components. In this case, the Session Menu Generator program creates the Session Menu Web page, and therefore has knowledge of it. The Session Menu Web page, on the other hand, has no knowledge of the generator itself.

Custom Icons. There are two different kinds of components in Figure A-5. To make them visually distinct, we extend UML with two new icons—one for CGI programs and one for HTML pages. Figure A-6 shows the components involved in Use Case #2: Enroll in a course. Web pages are drawn as pages with a ‘W’ in them. CGI programs are drawn as Circles with “CGI” in them.

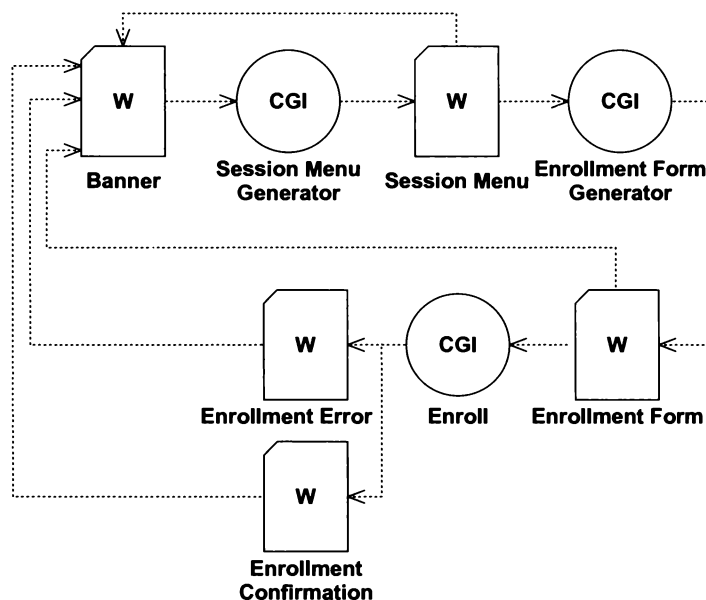


Figure A-6 Enrollment Components

Component Flow. Figure A-6 introduces two new Web pages and one new CGI program. We have decided that the application should begin with a banner page of some kind. Presumably, this page will have links for the various kinds of operations that users can perform. The Session Menu Generator is invoked by the banner page and generates the Session Menu page. Presumably the Session Menu page has links or buttons that allow the user to enroll in a course. The Session Menu invokes the Enrollment Form Generator CGI to create the form necessary for enrolling in the selected course. Once the user fills out this form, the Enroll CGI is invoked. This program validates and records the information on the form. If the form data are invalid, it generates the Enrollment Error page; otherwise it generates the confirmation page and sends the necessary e-mail messages. (Refer back to Use Case #2.)

Enhancing Flexibility. The astute reader will have recognized that there is a rather nasty inflexibility built into this component model. The CGI programs generate most of the Web pages. This means that the HTML text within the Web pages must be contained within the CGI programs. This makes modifying the Web pages a matter of modifying and rebuilding the CGI programs. We would rather the bulk of the generated Web pages were created with a nice HTML editor.

Thus, the CGI programs should read in a template of the Web pages that they are going to generate. The template should be marked with special flags, which will be replaced with the HTML that the CGI programs must generate. This means that the CGI programs share something in common. They all read template HTML files and add their own bits of HTML to them.⁶

Figure A-7 shows the resultant component diagram. Notice that I have added a WT icon. This represents an HTML template. It is a text file in HTML format with the special marks that the CGI programs use as insert points for the HTML that they generate. Notice also the direction of the dependency relationships between the CGI programs and the HTML templates. They might appear backwards to you. But remember they are *dependency* relationships, not data flows. The CGI programs know about (depend on) the HTML templates.

6. This chapter was written long before XSLT came on the scene. Nowadays, we'd likely solve the problem by having the CGI scripts (or servlets) generate XML and then invoke an XSLT script to translate that into HTML. On the other hand, generating HTML with XSLT still doesn't give us the ability to design our Web pages with a nice WYSIWYG editor. Sometimes I think the template scheme outlined in this chapter would be better in many instances.

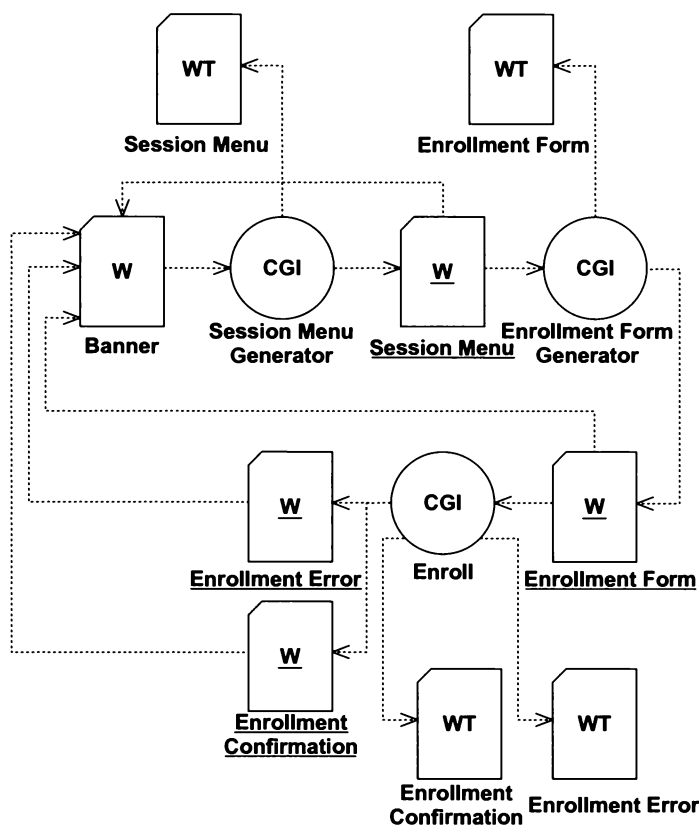


Figure A-7 Adding HTML templates to the Enrollment Component Diagram

The Specification/Instance Dichotomy. In Figure A-7, the names of the generated Web pages are underlined. This is because these pages only exist at run time. They are instances of the HTML templates. In UML, the convention is that we underline instances. An instance is a software element that is generated from a specification (source document) of some kind. We'll have more to say about this later. For now, simply realize that the elements whose names are not underlined represent elements that must be written by hand and that act as specifications. Elements whose names are underlined are the product of some process that generates them from those specifications.

Using the HTML Templates. The HTML templates provide a great deal of flexibility to the architecture of this application. How do they work? How do the CGI programs place their generated output in the proper places within the HTML Templates?

We might consider placing a special HTML tag in the HTML template files. Such a tag would mark the position in the generated HTML file where the CGI program would insert its output. However, these generated Web pages may have several sections, each of which might require its own insertion point into which the CGI can insert HTML. Thus, each HTML template could have more than one insertion tag, and the CGI will have to be able to somehow specify which output goes with which tag.

The tag could look like this: <insert name> where "name" is an arbitrary string that identifies the insertion point to the CGI. A tag such as <insert header> will allow a CGI to specify the name 'header' and then completely replace the tag with generated output.

Clearly each tag is replaced with a string of characters, so each tag represents a kind of named stream of characters. We can imagine the kind of C++ code that might exist in the CGI as follows:

```
HTMLTemplate myPage("mypage.htm");
myPage.insert("header",
    "<h1> this is a header </h1>\n");
cout << myPage.Generate();
```

This would send HTML to cout that was generated from the template mypage.html in which the tag <insert header> was replaced with the string "<h1> this is a header </h1>\n".

Figure A-8 shows how we might design the HTMLTemplate class. This class holds the name of the template file as an attribute. It also has methods that allow replacement strings to be inserted for specifically named insertion points. Instances of HTMLTemplate will contain a map that relates the insertion point name to the replacement string.

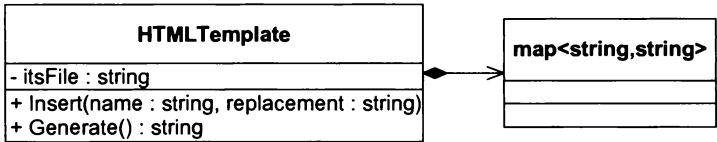


Figure A-8 HTMLTemplate design

Notation. This is our first true class diagram. It shows us two classes related with a *composition* relationship. The icon used for the HTMLTemplate class and the map<string,string> class is not new to us. It was explained in the sidebar named “Overview of UML Class Notation and Semantics” on page 473. The syntax of the attributes and operations is described below in the sidebar entitled, “Attributes and Operations.”

Attributes and Operations

Attributes and operations can be adorned with the following encapsulation specifiers:

- + Public
- Private
- # Protected

The type of an attribute can be specified as an identifier that follows the attribute and is separated from it with a colon (e.g., count : int).

Similarly, the types of the arguments of a function are specified using the same colon notation (e.g., SetName (name : string)).

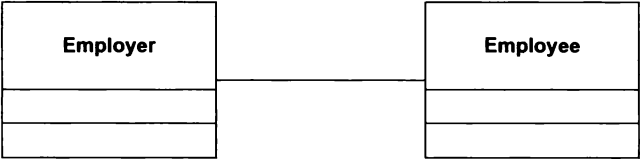
Finally, the return type of an operation can be specified with an identifier that follows the name and argument list of the operation and is separated from it by a colon (e.g., Distance(from : Point) : float).

The arrowhead on the association that connects the two classes in Figure A-8 indicates that HTMLTemplate knows about map<string,string>, but that the map does not know about the HTMLTemplate. The black diamond on the end of the association nearest the HTMLTemplate class identifies this as a special case of association called composition. (See the sidebar entitled, “Association, Aggregation, and Composition.”) It indicates that HTMLTemplate is responsible for the lifetime of the map class.

Association, Aggregation, and Composition

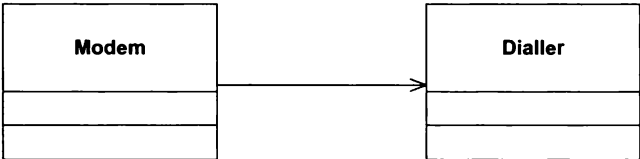
An association is a relationship between two classes that allows instances created from those classes to send messages to each other. (i.e., links may exist between objects whose classes are associated). It is denoted by a line that connects the two classes. Associations are most often implemented as instance variables in one class that point or refer to the other.

Association



The navigability of an association can be restricted by adding arrowheads to the associations. When an arrowhead is present, the association can only be navigated in the direction of the arrow. This means that the class to which the arrow points does not know about its associate.

Navigable Association



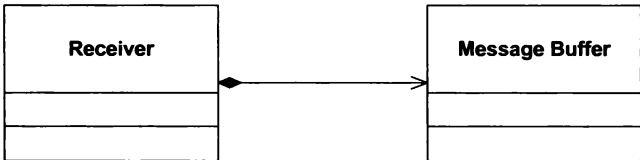
Aggregation is a special form of association. It is denoted with a white diamond on the aggregate class. Aggregation implies a “whole/part” relationship. The class adjacent to the white diamond is the “whole,” and the other class is its “part.” The “whole/part” relationship is purely connotative; there is no semantic difference from association^a.

Aggregation



Composition is a special form of aggregation. It is denoted with a black diamond. It implies that the “whole” is responsible for the lifetime of its “part.” This responsibility does not imply either creation or deletion responsibility. Rather, it implies that the “whole” must see to it that the “part” is somehow deleted. That can be accomplished by directly deleting the “part,” or by passing the “part” to another entity that assumes responsibility for it.

Composition



a. With one exception. Reflexive or cyclic aggregation relationships between *objects* is not allowed. That is, instances cannot participate in a cycle of aggregations. If this rule were not in place, all instances in the cycle would be part of themselves. That is, a part could contain its whole.

Note that this rule does not prevent *classes* from participating in a cycle of aggregations; it simply restricts their instances.

The Database Interface Layer. Each CGI programs must also have access to the data that represent the courses, classes, students, etc. We shall call this the training database. The form of this database is, as yet, undecided. It could be held in a relational database or in a set of flat files. We do not want the architecture of our application to depend on the form in which the data are stored. We would like the bulk of each application to remain unchanged when the form of the database changes. Thus, we will shield the application from the database by interposing a database interface layer (DIL).

In order to be effective, a DIL must have the special dependency characteristics shown in Figure A-9. The DIL depends on the application, and the DIL depends on the database. Neither the application nor the database has any knowledge of each other. This allows us to change the database without necessarily changing the application. It also allows us to change the application without changing the database. We can completely replace the database format or engine without affecting the application.

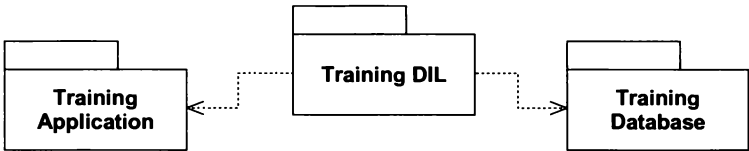
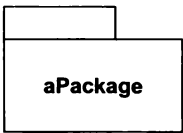


Figure A-9 Database Interface Layer Dependency Characteristics

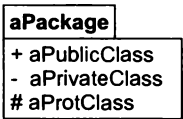
Notation. Figure A-9 shows a special kind of class diagram called a “package diagram.” The icons denote packages. Their shape is reminiscent of a file folder. Like a file folder, a package is a container. (See the “Packages and Subsystems” sidebar.) The packages in Figure A-9 contain software components such as classes, HTML files, CGI main program files, etc. The dashed arrow that connects the packages represents a *dependency* relationship. The arrowhead points at the target of the dependency. A dependency between packages implies that the dependent package cannot be used without the package on which it depends.

Packages and Subsystems

Packages are drawn as a large rectangle with a smaller rectangular “tab” on the upper left of the large rectangle. Normally, the name of the package is placed in the large rectangle.



Packages can also be drawn with the package name in the “tab” and the contents of the package in the large rectangle. The contents may be classes, files, or other packages. Each can be prefixed with the (–, +, #) encapsulation icons to denote that they are private, public, or protected within the package.

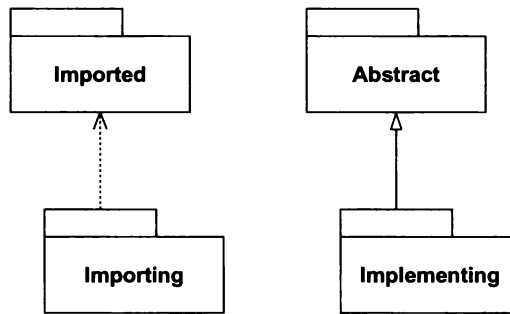


Packages may be connected by one of two different relationships. The dashed dependency arrow is called an *import dependency*. It is a dependency relationship with a stereotype of «import». This

(continues on next page)

stereotype is the default when a dependency is used with a package. The base of the arrow is attached to the importing package, and the arrowhead touches the imported package. An import dependency implies that the importing package has visibility to any of the public elements of the imported package. This means that the elements of the importing package can use any public element of the imported package.

Packages can also be connected with Generalization relationships. The open triangular arrowhead touches the general or abstract package, and the other end of the relationship touches the implementing package. The implementing package has visibility to any public or protected element of the abstract package.



There are several defined stereotypes of package. The default is «package», which denotes a container without any special constraints. It can hold anything that can be modeled in UML. Typically, it is used to denote a physical, *releaseable*, unit. Such a package would be tracked in a configuration management and version control system. It might be represented by subdirectories in a file system or by the module system of a language (e.g., Java packages or JAR files). Packages represent a partitioning of the system that enhances its developability and releasability.

The «subsystem» stereotype for packages denotes a logical element that, in addition to containing model elements, also specifies their behavior. A subsystem can be given operations. Those operations must be supported by use cases or collaborations within the package. Subsystems represent a *behavioral* partitioning of a system or application.



These two kinds of packages are orthogonal to each other. The partitioning that enhances developability and releasability is almost never similar to a partitioning that is based upon behavior. The former is often used by software engineers as the unit of configuration management and version control. The latter is used more often by analysts in order to describe the system in an intuitive fashion and to perform impact analyses when features change or are added.

Database Interface. The classes within the Training Application package need some way to access the database. This will be accomplished through a set of *interfaces* inside the Training Application package. (See Figure A-10.) These interfaces represent the types in the domain model of Figure A-4 on page 475. The interfaces are implemented by classes in the DIL package. They will be used by the other classes within the Training Application

package to access the data in the database. Notice that the direction of the dependencies in Figure A-10 corresponds to the direction of the import relationships between the packages in Figure A-9.

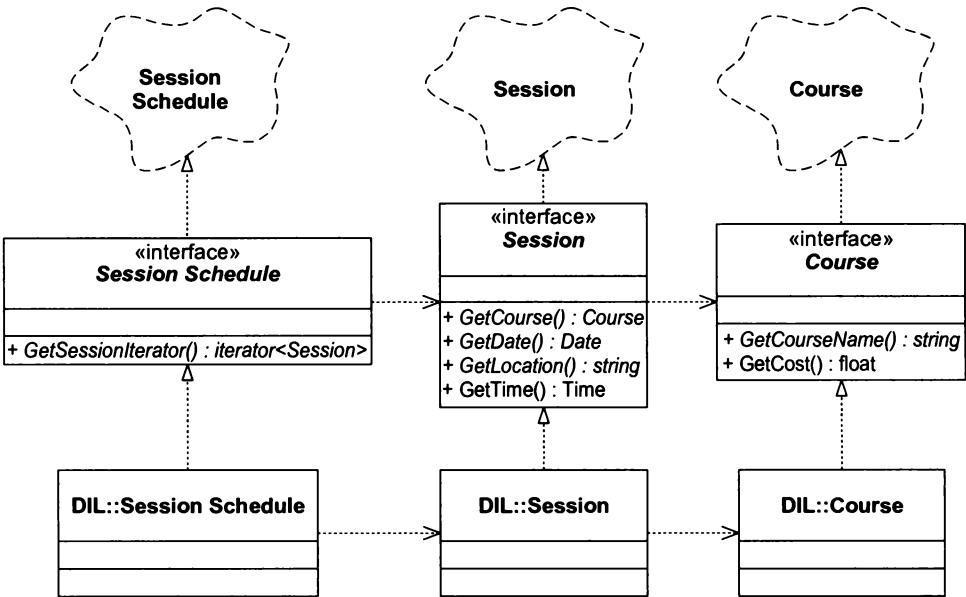


Figure A-10 Database interface classes in the Training Application package

Notation. When the name of a class is written in italics, it indicates that the class is abstract.⁷ Since interfaces are completely abstract classes, it is appropriate to italicize their names. The operations are also shown in italics to denote that they are abstract. The classes in the DIL are bound to the interfaces through *realizes* relationships, which are drawn as dashed lines with open triangular arrowheads pointing at the interfaces. In Java, these would represent “implements” relationships. In C++, they would represent inheritance.

Interfaces are physical structures. In languages like C++ and Java, they have source code counterparts. Types, on the other hand, are not physical and do not represent something that has a source-code equivalent. In Figure A-10, we have drawn realizes relationships from the interfaces to the types that they represent. This does not represent a physical relationship, nor is there any corresponding source code. In this case, the realizes relationship is showing the correspondence between the physical design entities and the domain model. This correspondence is seldom as clear-cut as depicted here.

The *imports* relationships between the packages in Figure A-9 are shown in Figure A-10 through the use of the double colon. The `DIL::Session` class is a class named `Session` that exists in the `DIL` package and is visible to (has been imported into) the `TrainingApplication` package. The fact that there are two classes named `Session` is acceptable since they are in different packages.

The Session Menu Generator. Referring back to Figure A-7, we see that the first CGI program is the `SessionMenuGenerator`. This program corresponds to the very first use case back on page 469. What does the design of this CGI program look like?

Clearly it must build an HTML representation of the session schedule. Thus, we will need an `HTMLTemplate` that merges the boiler plate of the session menu with the actual data from the session schedule. Also, the program will use the `SessionSchedule` interface to access the `Session` and `Course` instances in the database in order to get their names, times, locations, and fees. Figure A-11 shows a sequence diagram that describes the process.

7. An abstract class has at least one pure (or abstract) method.

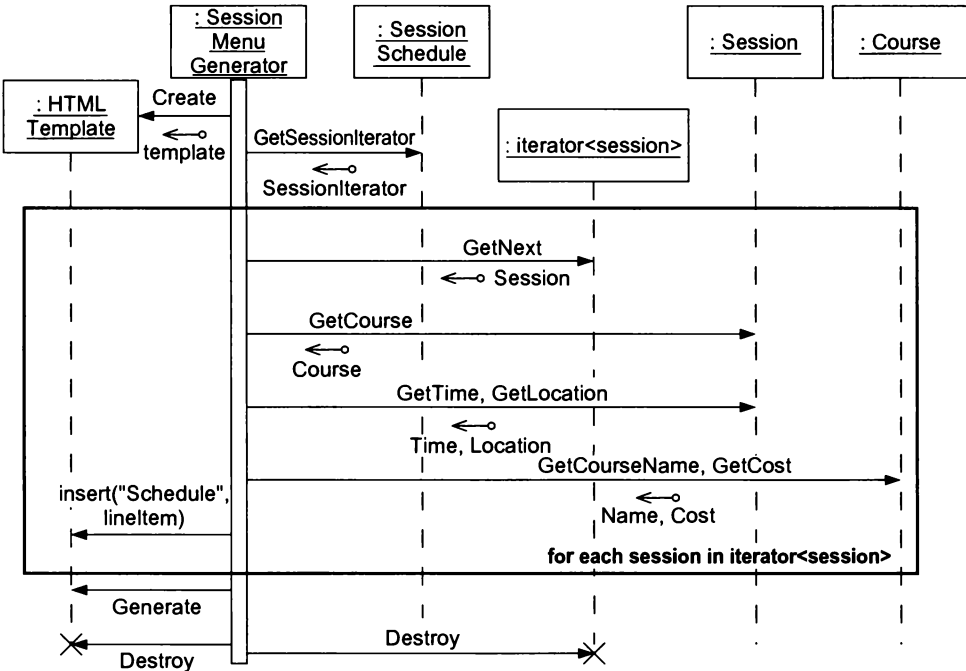


Figure A-11 SessionMenuGenerator Sequence Diagram

The SessionMenuGenerator object is created by main and controls the entire application. It creates an HTMLTemplate object passing it the name of the template file. It then fetches an iterator<Session>⁸ object from the SessionSchedule interface. It loops through every Session in the iterator<Session> asking each Session for its Course. It fetches the time and location of the course from the Session object, and the name and cost of the course from the Course object. The last action in the loop is to create a line item from all that information and Insert it into the HTMLTemplate object at the Schedule insertion point. When the loop is completed, the SessionMenuGenerator invokes the Generate method of the HTMLTemplate and then destroys it.

Notation. The names within the rectangles on the sequence diagram in Figure A-11 are underlined. This indicates that they represent objects rather than classes. Object names are composed of two elements separated by a colon. Prior to the colon is the *simple name* of the object. Following the colon is the name of a class or interface that the object implements. In Figure A-11, the simple names are all omitted, so all the names begin with a colon.

The dashed lines hanging down from the objects are called *lifelines*, and they represent the lifetime of the objects. All the objects in Figure A-11, except HTMLTemplate and iterator<Session>, show lifelines that start at the top and end at the bottom. By convention, this means that those objects were in existence before the scenario began and remain in existence when the scenario ends. HTMLTemplate, on the other hand, is explicitly created and destroyed by SessionMenuGenerator. This is evident by the arrow that terminates on HTMLTemplate, thus creating it, and the “X” that terminates its lifeline at the bottom. SessionMenuGenerator also destroys Iterator<Session>; however, it is not clear what object creates it. The creator is probably a derivative of SessionSchedule. Thus, while Figure A-11 does not explicitly show the creation of the Iterator<Session> object, the position of the start of the object’s lifeline implies that it is created at about the time that the GetSessionIterator message is sent to the SessionSchedule object.

The arrows that connect the lifelines are messages. Time proceeds from top to bottom, so this diagram shows the sequence of messages that are passed between the objects. The messages are named by labels close to the

8. This chapter was written before STL was commonplace. At the time I was using my own container library, which had templated iterators.

arrows. The short arrows with circles on the end are called *data tokens*. They represent data elements that are passed in the context of the message. If they point in the direction of the message, they are parameters of the message. If they point against the direction of the message, they are values returned by the message.

The long, skinny rectangle on the `SessionMenuGenerator` lifeline is called an *activation*. An activation represents the duration of the execution of a method or function. In this case, the message that started the method is not shown. The other lifelines in Figure A-11 do not have activations because the methods are all very short and do not emit other messages.

The bold rectangular box that surrounds some messages in Figure A-11 defines a loop. The completion criterion of the loop is mentioned at the bottom of the box. In this case, the enclosed messages will repeat until all the `Session` objects within `iterator<Session>` have been examined.

Note that two of the message arrows have been overloaded with more than one message. This is just a shorthand to minimize the number of arrows. The messages are sent in the order mentioned, and the return values come back in the same order.

Abstract Classes and Interfaces in Sequence Diagrams

The astute reader will notice that some of the objects in Figure A-11 are instantiated from interfaces. `SessionSchedule`, for example, is one of the database interfaces classes. This may seem to be violating the principle that objects cannot be instantiated from abstract classes or interfaces.

The class name of an object in a sequence diagram does not need to be the name of the actual type of the object. It is sufficient that the object simply conform to the interface of the named class. In a static language like C++, Java, or Eiffel, the object should belong either to the class named in the sequence diagram or to a class that derives from the class or interface in the sequence diagram. In dynamic languages like Smalltalk or Objective-C, it is sufficient that the object conform to the interface named in the sequence diagram.⁹

Thus, the `SessionSchedule` object in Figure A-11 refers to an object whose class implements or derives from the `SessionSchedule` interface.

Static Model of the Session Menu Generator. The dynamic model shown in Figure A-11 implies the static model shown in Figure A-12. Note that the relationships are all either dependencies or stereotyped associations. This is because none of the classes shown in the figure hold instance variables that refer to the others. All the relationships are transient in that they do not outlast the execution of the activation rectangle on the `SessionMenuGenerator` lifeline of Figure A-11.

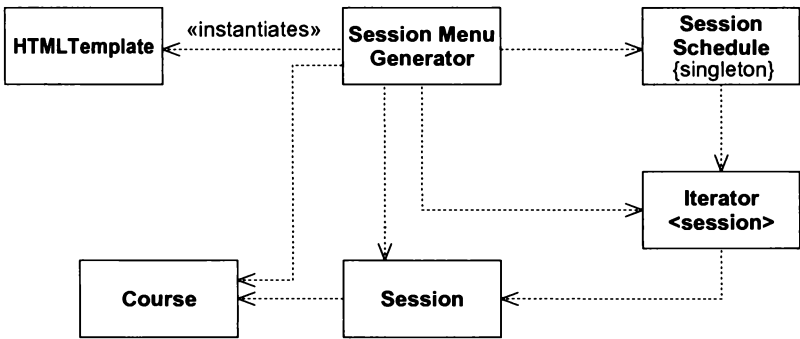


Figure A-12 Static Model of the Session Menu Generator application

9. If you don't understand this, don't worry about it. In dynamic languages like Smalltalk and Objective-C, you can send any message you like to any object you like. The compiler does not check to see whether the object can accept that message. If, at run time, a message is sent to an object that does not recognize it, a run-time error will occur. Thus, it is possible for two completely different and unrelated objects to accept the same messages. Such objects are said to conform to the same interface.

The relationship between `SessionMenuGenerator` and `SessionSchedule` deserves special mention. Notice that `SessionSchedule` is shown with the `{singleton}` property. This denotes that only one `SessionSchedule` object can exist within the application and that it is accessible at the global scope. (See `SINGLETON` on page 178.)

The dependency between `SessionMenuGenerator` and `HTMLTemplate` carries the stereotype `«creates»`. This simply indicates that the `SessionMenuGenerator` instantiates `HTMLTemplate` instances.

The associations with `«parameter»` stereotypes show that the objects find out about each other through method arguments or return values.

How Does the `SessionMenuGenerator` Object Get Control? The activation rectangle on `SessionMenuGenerator`'s lifeline in Figure A-11 does not show how it was started. Presumably, some higher level entity, like `main()`, called a method on `SessionMenuGenerator`. We might call this method `Run()`. This is interesting because we have several other CGI programs to write, and they will all need to be started by `main()` somehow. Perhaps there is a base class or interface called `CGIProgram` that defines the `Run()` method, and perhaps `SessionMenuGenerator` derives from it.

Getting the User's Input into the CGI Program. The `CGIProgram` class will help us with another issue. CGI programs are typically invoked by a browser after the user has filled out a form on the browser screen. The data entered by the user are then passed to the `main()` function of the CGI program through standard input. Thus, `main()` could pass a reference to the standard input stream to the `CGIProgram` object, which would in turn make the data conveniently available to its derivatives.

What is the form of the data passed from the browser to the CGI program? It is a set of name-value pairs. Each field in the form that was filled out by the user is given a name. Conceptually, we would like the derivatives of `CGIProgram` to be able to ask for the value of a particular field by simply using its name. For example,

```
string course = GetValue("course");
```

Thus, `main()` creates the `CGIProgram` and primes it with the needed data by passing the standard input stream into its constructor. The `main()` function then calls `Run()` on the `CGIProgram`, allowing it to begin. The `CGIProgram` derivative calls `GetValue(string)` to access the data in the form.

But this leaves us with a dilemma. We'd like to make the `main()` function generic, yet it must create the appropriate derivative of `CGIProgram` and there are many such derivatives. How can we avoid having multiple `main()` functions?

We can solve this by using link-time polymorphism. That is, we implement `main()` in the implementation file of the `CGIProgram` class (i.e., `cgiProgram.cc`). In the body of `main()`, we declare a global function named `CreateCGI`. However, we do not implement the function. Rather, we will implement this function in the implementation file of the derivative of `CGIProgram` (e.g., `sessionMenuGenerator.cc`). (See Figure A-13.)

Authors of CGI programs no longer need to write a `main()` program. Rather, in each derivative of `CGIProgram`, they must supply the implementation for the global function `CreateCGI`. This function returns the derivative back to `main()`, which can then manipulate it as necessary.

Figure A-13 shows how we use components with the `«function»` stereotype to represent free global functions. The figure also demonstrates the use of properties to show what files the functions are implemented in. Notice that the `CreateCGI` function is annotated with the property `{file=sessionMenuGenerator.cc}`.

Summary

In this chapter, we have walked through a large portion of the UML notation in the context of a simple example. We have shown the various notational conventions used in the different phases of software development. We showed how a problem can be analyzed using use cases and types to form an application domain model. We

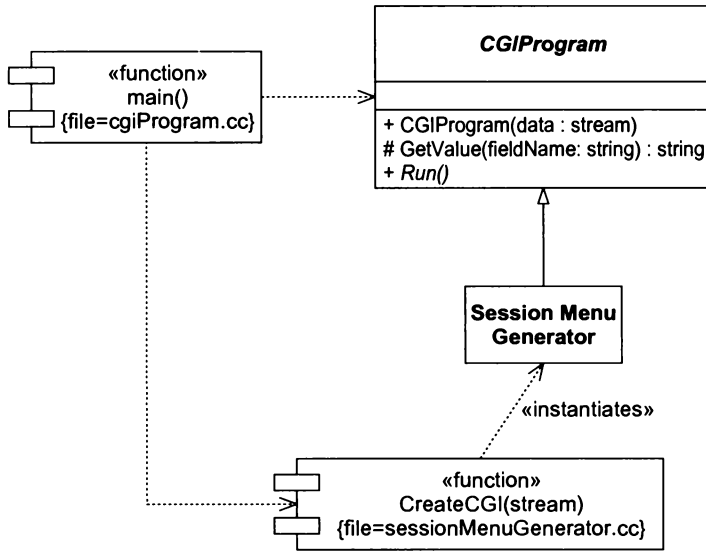


Figure A-13 Architecture of CGI programs

showed how classes, objects, and components can be combined into static and dynamic diagrams to describe the architecture and construction of the software. We showed the UML notation for each of these concepts and demonstrated how the notation can be used. And, by no means of least importance, we showed how all these concepts and notations participate in software-design reasoning.

There is more to learn about UML and software design. The next chapter will use another example to explore more of UML and different analysis and design tradeoffs.

Bibliography

1. Booch, Grady. *Object Oriented Analysis and Design with Applications*, 2nd ed. Benjamin Cummings: 1994.
2. Rumbaugh, et al. *Object Oriented Modeling and Design*. Prentice Hall: 1991.
3. Wirfs-Brock, Rebecca, et al. *Designing Object-Oriented Software*. Prentice Hall: 1990.
4. Coad, Peter, and Ed Yourdon. *Object Oriented Analysis*. Yourdon Press: 1991.
5. Jacobson, Ivar. *Object Oriented Software Engineering a Use Case Driven Approach*. Addison-Wesley, 1992.
6. Cockburn, Alistair. *Structuring Use Cases with Goals*. <http://members.aol.com/acockburn/papers/usecass.htm>.
7. Kennedy, Edward. *Object Practitioner's Guide*. <http://www.zoo.co.uk/~z0001039/PracGuides>. November 29, 1997.
8. Booch, Grady. *Object Solutions*. Addison-Wesley, 1995.
9. Gamma, et al. *Design Patterns*. Addison-Wesley, 1995.

APPENDIX B

UML Notation II: The STATMUX

In this chapter, we continue our exploration of the UML notation, this time focussing on some of its more detail-oriented aspects. As a context for this exploration, we will study the problem of a statistical multiplexor.

The Statistical Multiplexor Definition

A statistical multiplexor is a device that allows multiple serial data streams to be carried over a single telecommunications line. Consider, for example, a device that contains a single 56K modem and has 16 serial ports. When two such devices are connected together over a phone line, characters that are sent into port 1 on one device come out of port 1 on the other device. Such a device can support 16 simultaneous full duplex communications sessions over a single modem.

Figure B-1 shows a typical 1980s application for such a device. In Chicago, we have a mix of ASCII terminals and printers that we want to connect to a VAX in Detroit. We have a leased 56K line that connects the two locations. The statistical multiplexor creates 16 virtual serial channels between the two locations.

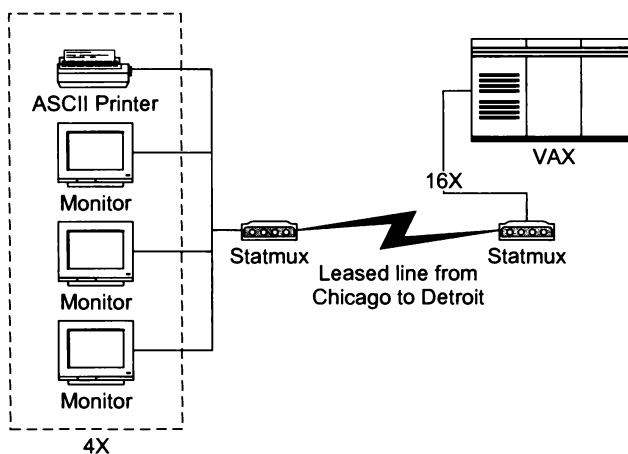


Figure B-1 Typical Statistical Multiplexor Application

Clearly, if all 16 channels are running simultaneously, they will all distribute the 56K throughput between them, yielding an effective bit rate of somewhat less than 3500 bits per second per device. However, most terminals and printers are not busy 100% of the time. Indeed, the duty cycle in many applications is well under 10%. Thus, though the line is shared, statistically each user will perceive near 56K performance.

The problem we are going to study in this chapter is the software inside the statmux. This software controls the modem and serial-port hardware. It also determines the multiplexing protocol used to share the communications line between all the serial ports.

The Software Environment

Figure B-2 is a block diagram¹ that shows where the software fits in the statmux. It sits between the 16 serial ports and the modem.

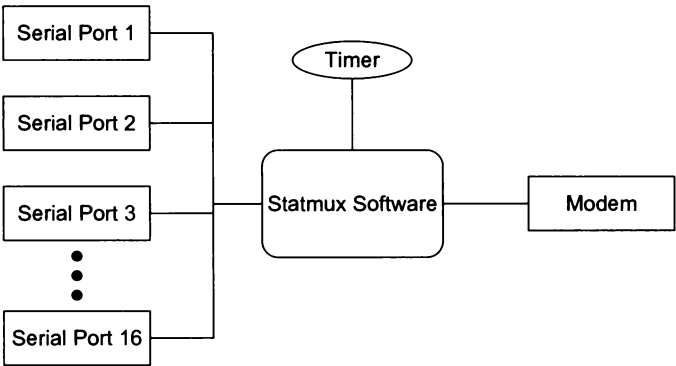


Figure B-2 Statmux system block diagram

Each serial port generates two interrupts to the main processor—one when it is ready to send a character and one when a character has been received. The modem also generates similar interrupts. Thus, there are 34 interrupts coming into the system. The modem interrupts are higher priority than the serial-port interrupts. This ensures that the modem can run at the full 56K speed even if the other serial ports must suffer periods of dormancy.

Finally, there is a timer that generates an interrupt every millisecond. This timer allows the software to schedule events for particular times.

The Real-time Constraints

A little calculation will demonstrate the problem that this system faces. At any given time, 34 interrupts sources may be demanding service at a rate of 5600 interrupts per second—plus an extra thousand interrupts per second from the timer. This amounts to 191,400 interrupts per second. Thus, the software can spend no more than 5.2 μ s servicing each interrupt. This is very tight, and we will need a reasonably fast processor to make sure that we don't drop any characters.

To make matters worse, the system has more work to do than simply servicing the interrupts. It also has to manage the communications protocol across the modem, gather the incoming characters from the serial ports, and divvy out the characters that need to be sent to the serial port. All of this is going to require some processing that must somehow fit between the interrupts.

Fortunately, the maximum sustained throughput of the system is only 11,200 characters per second (i.e., the number of characters that can be simultaneously sent and received by the modem). This means that, on average, we have nearly 90 μ s between characters.²

1. Block diagrams are a form of Kent Beck's GML (galactic modeling language). GML diagrams are composed of lines, rectangles, circles, ovals, and any other shape necessary to get the point across.

2. An eternity.

Since our ISRs (interrupt service routines) cannot exceed 5.2 μ s in duration, we have at least 94% of the processor still available to us between interrupts. This means we don't need to be overly concerned about efficiency outside of the ISRs.

The Input Interrupt Service Routine

These may have to be written in assembly language. The prime goal of the input ISRs is to get the character from the hardware and store it somewhere where the non-ISR software can deal with it at its leisure. The typical way of dealing with this is to use a ring buffer.

Figure B-3 presents a class diagram that shows the structure of the input ISRs and their ring buffers. We have invented a few new stereotypes and properties to describe the rather unique issues involved with interrupt service routines.

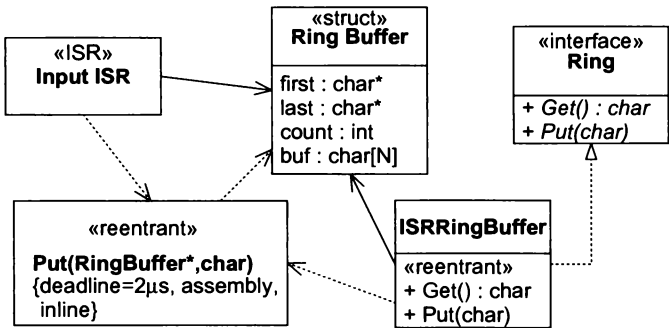


Figure B-3 The Input Service Interrupt Routine

First we have the InputISR class. This class has the stereotype «ISR,» which indicates that the class is an interrupt service routine. Such classes are written in assembly language and have only one method. This method has no name and is invoked only when the interrupt occurs.

InputISR has an association to its particular RingBuffer. The «struct» stereotype on the RingBuffer class indicates that this is a class with no methods. It is nothing more than a data structure. We have done this because we expect it to be accessed by assembly language functions that would not otherwise have access to the methods of a class.

The Put(RingBuffer*, char) function is shown in a class icon with the stereotype of «reentrant». This stereotype, used in this way, represents a free function that is written to be safe from interrupts.³ This function adds characters to the ring buffer. It is called by InputISR when a character has been received.

The properties on the function indicate that it has a real-time deadline of 2 μ s, that it should be written in assembly language, and that it should be coded inline wherever it is invoked. These last two properties are an attempt to meet the first.

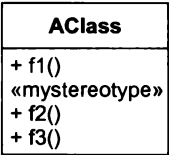
The ISRRingBuffer class is a regular class whose methods run outside of interrupt service routines. It makes use of the RingBuffer struct and provides a class facade for it. Its methods all conform to the «reentrant» stereotype and are therefore interrupt safe.

The ISRRingBuffer class realizes the Ring interface. This interface allows the clients outside the interrupt service routines to access the characters stored in the ring buffers. This class represents the interface boundary between the interrupts and the rest of the system.

3. Reentrancy is a complex topic that is beyond the scope of this chapter. The reader is referred to good texts on real-time and concurrent programming such as Doug Lea's *Concurrent Programming in Java*, Addison-Wesley, 1997.

Stereotypes in List Boxes

When stereotypes appear in the list compartments of classes, they have a special meaning. Those elements that appear below the stereotype conform to that stereotype.



In this example, function f1() has no explicit stereotype. Functions f2() and f3(), however, conform to the «mystereotype» stereotype.

There is no limit on the number of stereotypes that can appear in a list box like this. Each new stereotype overrides the previous. All elements shown between two stereotypes conform to the stereotype above them.

An empty stereotype «» can be used in the midst of a list to show that the following elements have no explicit stereotype.

Ring Buffer Behavior. The ring buffers can be described by a “simple” state machine as shown in Figure B-4. The state machine shows what happens when the Get () and Put () methods are called on objects of the ISRRingBuffer class.

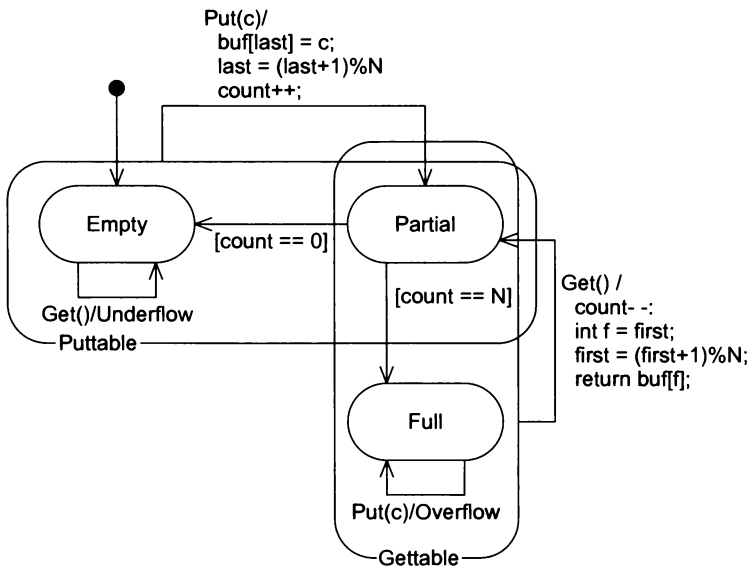


Figure B-4 Ring Buffer State Diagram

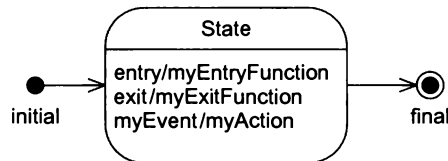
The diagram shows the three states that a ring buffer can be in. The machine begins in the Empty state. In the Empty state, the ring buffer has no characters in it. In that state, the Get () method will cause an underflow. We have not here defined what happens during underflow or overflow—those decisions are left for later. The two states

Empty and Partial are substates of the Puttable superstate. The Puttable superstate represents those states in which the Put() method operates without overflowing. Whenever the Put() method is invoked from one of the Puttable states, the incoming character is stored in the buffer, and the counts and indexes are adjusted appropriately. The Partial and Full states are both substates of the Gettable superstate. The Gettable superstate represents those states in which the Get() function operates without underflowing. When Get() is called in those states, the next character is removed from the ring, and is returned to the caller. The counts and indexes are adjusted appropriately. In the Full state, the Put() function results in an Overflow.

The two transitions out of the Partial state are governed by *guard* conditions. The machine transitions from the Partial to the Empty state whenever the count variable goes to zero. Likewise, the machine transitions from the Partial state to the Full state whenever the count variable reaches the size of the buffer (N).

States and Internal Transitions

In UML, a state is represented by a rectangle with rounded corners. This rectangle may have two compartments.



The top compartment simply names the state. If no name is specified, then the state is anonymous. All anonymous states are distinct from one another.

The bottom compartment lists the internal transitions for the state. Internal transitions are denoted as *eventName/action*. The *eventName* must be the name of an event that can occur while the machine is in the given state. The machine responds to this event by remaining in the state and executing the specified *action*.

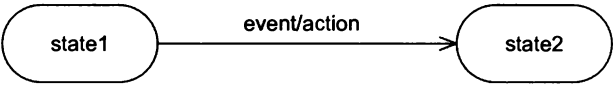
There are two special events that can be used in internal transitions. They are depicted in the icon above. The *entry* event occurs when the state is entered. The *exit* event occurs when the state is exited (even for a transition that returns immediately to that state).

An action may be the name of another finite-state machine that has both an initial and final state. Or an action can be a procedural expression written in some computer language or pseudocode. The procedure may use operations and variables of the object, if any, that contain the state machine. Or, the action can be of the form *^object.message(arg1, arg2, ...)*, in which case the action causes the named message to be sent to the named object.

There are two special *pseudostate* icons shown in the previous diagram. On the left, we see the black circle that represents the initial pseudostate. On the right, we see the bull's-eye that represents the final pseudostate. When a finite-state machine is initially invoked, it makes a transition from the initial pseudostate to the state to which it is connected. Thus, the initial pseudostate may only have one transition leaving it. When an event causes a transition to the final pseudostate, the state machine shuts down and accepts no more events.

Transitions between States

A finite-state machine is a network of states connected by transitions. Transitions are arrows that connect one state to another. The transition is labeled with the name of the event that triggers it.



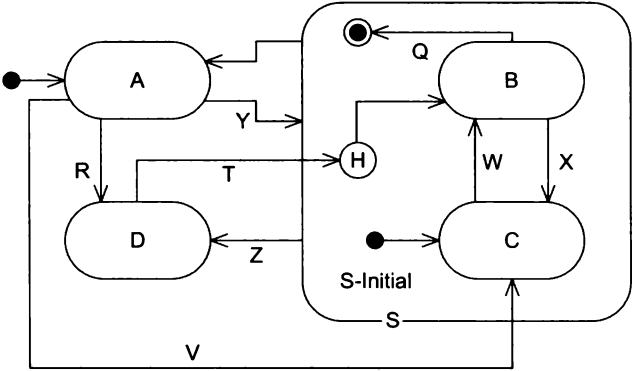
Here we see two states connected by a single transition. The transition will be ‘fired’ if the machine is in state1 and the event occurs. Upon the ‘firing’ of the transition, state1 is exited, and any exit action is executed. Then the action on the transition is executed. Then state2 is entered, and its entry action is executed.

An event on a transition can be qualified with a guard condition. The transition will only fire if the event occurs *and* the guard condition is true. Guard conditions are boolean expressions that appear in square brackets after the event name (e.g., myEvent [myGuardCondition]).

Actions on transitions are exactly the same as actions on internal transitions within states. (See “States and Internal Transitions.”)

Nested States

When one state icon completely encloses one or more others, the enclosed states are said to be *sub-states* of the enclosing *superstate*.



In the diagram above, states B and C are substates of the superstate S. The state machine begins in state A, as shown by the initial pseudostate. If transition V fires, then substate C within superstate S will become active. This will cause the entry functions of both S and C to be invoked.

If transition Y fires while in state A, the machine enters superstate S. A transition into a superstate must result in one of its substates becoming active. If a transition terminates at the edge of a superstate, as transition Y does, then there is a transition from the initial pseudostate within the superstate. Thus, transition Y triggers the transition from the S-Initial pseudostate to substate C.

(continues on next page)

When transition Y is fired, superstate S and substate C are both entered. Any entry actions for S and C are invoked at that time. The superstate's entry actions are invoked before the substate's entry actions. Transitions W and X may now fire, moving the machine between the B and C substates. The exit and entry actions will be performed as usual, but since superstate S is not exited, its exit functions will not be invoked.

Eventually, transition Z will fire. Notice that Z leaves the edge of the superstate. This means that, regardless of whether substate B or C is active, transition Z moves the machine to state D. This is equivalent to two separate transitions, one from C to D and the other from B to D, both labelled Z. When Z fires, the appropriate substate exit action is performed, and then the exit action for superstate S is performed. Then D is entered, and its entry action is performed.

Notice that transition Q terminates on a final pseudostate. If transition Q fires, then superstate S terminates. This will cause the unlabeled transition from S to A to fire. Terminating the superstate also resets any history information, as described later.

Transition T terminates on a special icon within the S superstate. This is called the *history marker*. When transition T fires, the substate that was last active within S becomes active again. Thus, if transition Z had occurred while C was active, then transition T will cause C to become active again.

If transition T fires when the history marker is inactive, then the unlabeled transition from the history marker to substate B fires. This denotes a default when there is no history information available. The history marker is inactive if S has never been entered or just after S has been terminated by transition Q.

Thus the event sequence Y-Z-T will leave the machine in substate C. But both R-T and Y-W-Q-R-T will leave the machine in substate B.

The Output Service Interrupt Routine

The processing for output interrupts is very similar to the processing for input interrupts. However, there are some differences. The noninterrupt parts of the system load up the output ring buffer with characters to be sent. The serial port generates an interrupt whenever it is ready for the next character. The interrupt service routine grabs the next character from the ring buffer and sends it to the serial port.

If there are no characters waiting in the ring buffer when the serial port becomes ready, then the interrupt service routine has nothing to do. The serial port has already signalled its readiness to accept a new character, and it will not do so again until it is given a character to send and finishes sending it. Thus, the flow of interrupts stops. We therefore need a strategy for restarting the output interrupts when new characters arrive. Figure B-5 shows the structure of the output ISR. Its similarity to the input ISR in Figure B-3 is obvious. However, notice the «calls» dependency from the `ISRRingBuffer` to the `OutputISR`. This indicates that the `ISRRingBuffer` object can cause the `OutputISR` to execute just as though an interrupt had been received.

Figure B-6 shows the necessary modifications to the finite-state machine of the output ring buffers. Compare this with Figure B4. Notice that the `Puttable` superstate has been removed and that there are two `Put` transitions. The first `Put` transition goes from the `Empty` state to the `Partial` state. This will cause the entry action of the `Gettable` superstate to be executed, which generates an artificial interrupt to the `OutputISR`.⁴ The second `Put` transition is internal to the `Partial` state.

4. The mechanism for generating an artificial interrupt of this kind is strongly dependent on the platform. On some machines, it is possible to simply call the ISR as though it were a function. Other machines require more elaborate protocols for artificially invoking ISRs.

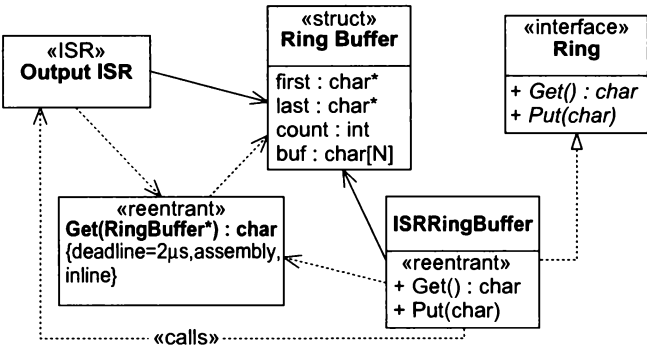


Figure B-5 Output Interrupt Service Routine

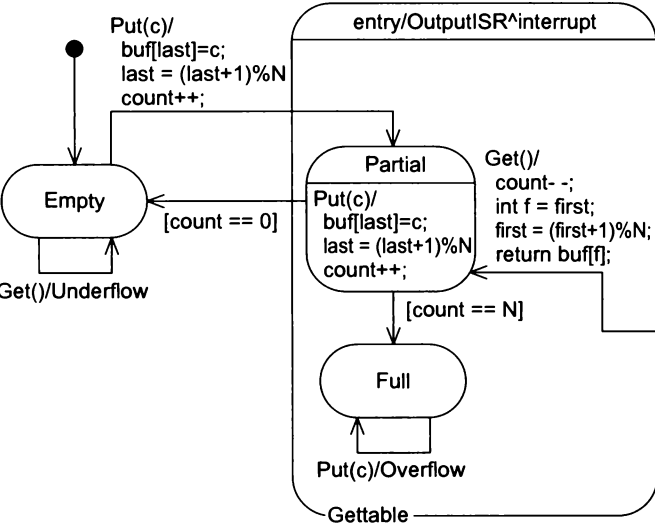


Figure B-6 Output Service Interrupt State machine

The Communications Protocol

Two statmuxes communicate via their modems. Each sends packets to the other over a telecommunications line. We must assume that the line is imperfect and that errors and dropouts can occur. Characters can be lost or garbled during transmission, and spurious characters may be created by electrical discharges or other electromagnetic interference. Therefore, a communications protocol must be put in place between the two modems. This protocol must be able to verify that packets are complete and accurate, and it must be able to retransmit packets that were garbled or lost.

Figure B-7 is an *activity diagram*. (See the “Activity Diagrams” sidebar on page 498.) It shows the communications protocol that our statmuxes will be using. This protocol is a relatively straightforward sliding window protocol with pipelining and piggybacking.⁵

The protocol begins at the initial pseudostate by initializing some variables and then creating three independent threads. The variables will be explained later, in the sections regarding the threads that depend on them. The “timing thread” is used to retransmit packets if no acknowledgment has been received within the allowed time period. It is also used to make sure that acknowledgments of properly received packets are sent in a timely fashion. The “sending thread” is used to send packets that have been queued for transmission. The “receiving thread” is used to receive, validate, and process packets. Let’s examine each in turn.

5. See *Computer Networks*, 2d. ed. Tanenbaum. Prentice Hall, 1988, Sec. 4.4 for more information about this kind of communications protocol.

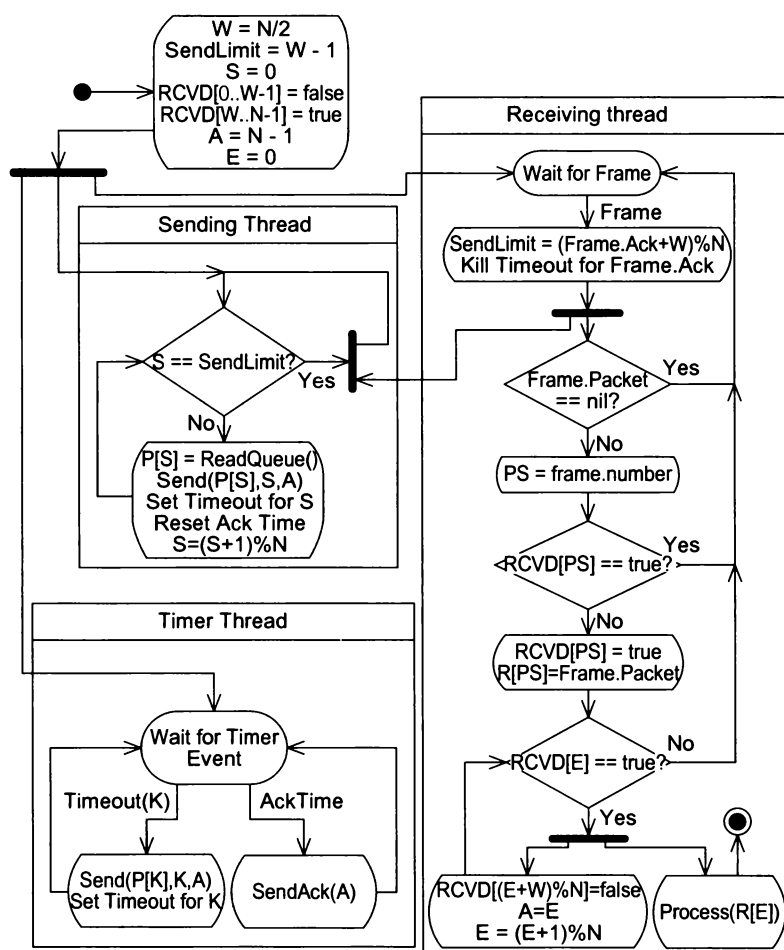


Figure B-7 Communications Protocol Activity Diagram

The Sending Thread. The s variable contains the serial number that will be stamped on the next outgoing packet. Each packet is numbered with a serial number in the range $0 \dots N$. The sending thread will continue to send packets, without waiting for them to be acknowledged, until there are w outstanding unacknowledged packets. w is set to $N/2$ so that never more than half the serial numbers are in play at any given time. The `SendLimit` variable normally holds the serial number of the packet that is beyond the window (w), and so it is the smallest serial number that cannot yet be sent.

As the sending thread continues to send packets, it increments s modulo N . When s reaches the `SendLimit`, the thread blocks until `SendLimit` is changed by the receiving thread. If s has not yet reached the `SendLimit`, then a new packet is pulled down from the queue by the `ReadQueue()` function. The packet is placed in the s position of the P array. We keep the packet in this array in case it needs retransmission. Then the packet is sent along with its serial number (s) and with the piggybacked acknowledgment (A). A is the serial number of the last packet that we received. This variable is updated by the receiving thread.

Having sent the packet, we start a time-out for it. If this time-out occurs before the packet is acknowledged, the timer thread will assume that either the packet or the acknowledgment was lost, and it will retransmit the packet.

At this point, we also reset the Ack timer. When the Ack timer expires, the timer thread assumes that too much time has elapsed since the last time we sent an acknowledgment, so it acknowledges the last-known good packet to be received.

The Receiving Thread. This thread begins by initializing a few variables. `RCVD` is an array of boolean flags indexed by serial number. As packets are received, they are marked in `RCVD` as true. Once a packet is processed, the serial number that is `w` past that packet is marked false. `E` is the serial number of the packet we are expecting and is next to be processed; it will always be `A+1` modulo `N`. As part of initialization, we set the last half of `RCVD` to true to denote the fact that those serial numbers are outside the allowed window. If they were to be received, they would be dropped as duplicate packets.

The receiving thread waits for a frame. A frame may be a packet or just a plain acknowledgment. In either case, it will contain the acknowledgment of the last good packet. We update the `SendingLimit` and trigger the sending thread. Notice that the `SendingLimit` is set to `w` past the last acknowledged frame. Thus, the sender is allowed to use only half the serial number space starting at the last acknowledged packet, so that the sender and receiver have negotiated which half of the serial number space is currently valid.

If the frame contains a packet, then we get the serial number of that packet and check the `RCVD` array to see if we have already received a packet with this serial number. If so, we drop it as a duplicate. Otherwise, we update the `RCVD` array to show that the packet has now been received, and we save the packet in the `R` array.

Although the packets are sent in serial number order, they can be received out of order. This is true simply because packets can get lost and retransmitted. Thus, even though we have just received packet number `PS`, it may not be the one we were expecting (`E`). If it is not, we simply wait until `E` is received. However, if `PS == E`, then we spawn off a separate thread to process the packet. We also move the allowed serial number window by setting the `E+W` slot of the `RCVD` array to false. Finally, we set `A` to `E` to signify that `E` was the last good received serial number, and we increment `E`.

The Timing Thread. The timer simply waits for a timer event. There are two kinds of events that can occur. A `Timeout(K)` event signifies that a packet was sent by the sending thread, but no acknowledgment was ever received. Thus, the timer thread retransmits the `K` packet and restarts its timer.

The `AckTime` event is generated by a repeating, retriggerable timer. This timer sends the `AckTime` event every `X` milliseconds. However, it can be retriggered to start over at `X`. The sending thread retriggers this timer every time it sends a packet. This is appropriate because each packet carries a piggyback acknowledgment. If no packet has been sent for `X` milliseconds, then the `AckTime` event will occur, and the timer thread will send an acknowledgment frame.

Whew! You may have found this discussion to be a bit of a struggle to get through. Just imagine what it would have been like without the explanatory text. The diagram may express my intent, but the extra words sure help. Diagrams can seldom stand on their own.

How do we know that the diagram is correct? We don't! It won't surprise me in the slightest if several readers find problem with it. Diagrams can't usually be tested directly the way code can. So we'll have to wait for the code to know if this algorithm is really correct.

These two issues make the utility of such diagrams questionable. They can make good pedagogical tools, but one should not be considered expressive and accurate enough to be the sole specification of a design. Text, code, and tests are also required.

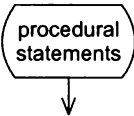
Activity Diagrams

Activity diagrams are a hybrid of state transition diagrams, flowcharts, and petri nets. They are especially good at describing event-driven multithreaded algorithms.

An activity diagram *is* a state diagram. It is still a graph of states connected by transitions. However, in an activity diagram, there are special kinds of states and transitions.

(continues on next page)

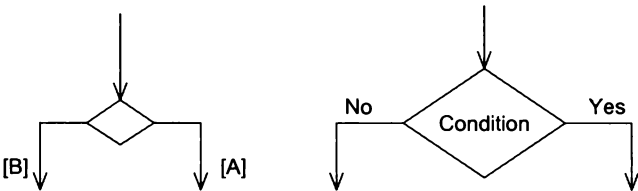
Action State



An *action state* is drawn as a rectangle with a flat top and bottom and rounded sides. This icon is distinct from a normal state icon in that the corners are sharp, whereas a state icon has rounded corners. (See “States and Internal Transitions” on page 493.) The inside of the action state contains one or more procedural statements that represent its entry actions. (This is just like a process box in a flowchart)

When an action state is entered, its entry actions are executed immediately. Once those actions are completed, the action state is exited. The outgoing transition must not have an event label, since the “event” is simply the completion of the entry actions. However, there may be several outgoing transitions, each with a mutually exclusive guard condition. The union of all the guard conditions must always be true (i.e., it is impossible to get “stuck” in an action state).

Decisions

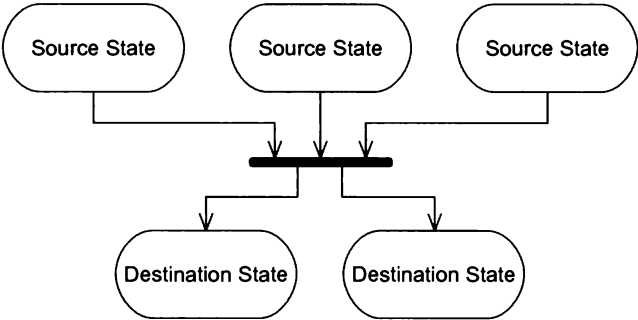


The fact that an action state can have many outgoing guarded transitions means that it can act as a decision step. However, it is often better to specifically denote decisions, for which the diamond icon is traditional.

A transition enters a diamond, and *N* guarded transitions leave it. Once again, the boolean union of all the outgoing guards must yield truth.

In Figure B-7, we used a variation of the diamond that is more akin to flowcharts. A boolean condition is stated within the diamond, and two outgoing transitions are labeled “Yes” and “No.”

Complex Transitions



Complex transitions show the splitting and joining of multiple threads of control. They are denoted by the dark bar called an *asynchronization* bar. Arrows connect states to the asynchronization bar. The states that lead to the bar are called *source* states, and the states leading away from the bar are called *destination* states.

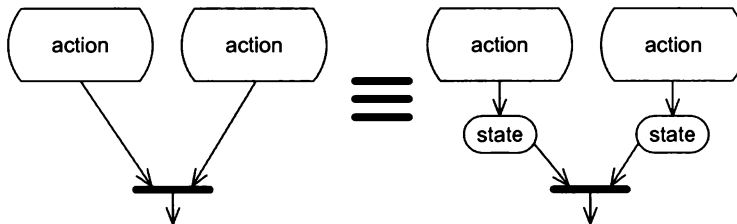
(continues on next page)

The entire group of arrows leading to and from the bar form, a *single transition*. The arrows are labeled neither with events nor with guards. The transition fires when all the source states are occupied (i.e., when the three independent threads are in the appropriate states). Moreover, the source states must be true states and not action states (i.e., they must be able to wait).

Upon firing, the source states are all exited, and the destination states are all entered. If there are more destination states than source states, then we have spawned new threads of control. If there are more source states, then some threads have joined.

Each time a source state is entered, the entry is counted. Each time the complex transition fires, the counters in its source states are decremented. A source state is considered to be occupied as long as the counter is nonzero.

As a notational convenience, a true transition or an action state can be used as a source for an asynchronization bar. (See Figure B-7.) In such a case, it is assumed that the transition actually terminates on a true unnamed state, which is a source state of the bar.



Structure of the Communications Protocol Software. The three threads of control all share the same variables. Thus, the functions invoked by those threads should probably be methods of the same class. However, most threading systems nowadays equate a thread with an object. That is, each thread has an object that controls it. In UML, these are called *active objects*. (See “Active Objects” on page 502.) So the class that houses the protocol methods will also need to create the active objects that control the threads.

The timer thread will be useful in places other than the protocol, so its thread should probably be created in a different part of the system. This leaves the sending thread and the receiving thread to be created by the protocol object.

Figure B-8 shows an object diagram (see “Object Diagrams” on page 501) that depicts the situation just after the `CommunicationsProtocol` object has been initialized. The `CommunicationsProtocol` has created two `Thread` objects and keeps responsibility for their lifetimes. The `Thread` objects employ the `COMMAND`⁶ pattern in order for the newly created thread of execution to get started. Each `Thread` holds an instance of an object that conforms to the `Runnable` interface. (See “Interface Lollipops” on page 502.) The `ADAPTER`⁷ pattern is then used to bind the `Threads` to the appropriate methods of the `CommunicationsProtocol` object.

A similar arrangement can be seen between the `Timer` and the `CommunicationsProtocol`. However, in this case the lifetime of the `Timer` object is not controlled by the `CommunicationsProtocol` object.

The «friend» relationships exist because we want the methods invoked by the adapters to be private to `CommunicationsProtocol`. We don’t want them to be called by anyone other than the adapters.⁸

6. page 151

7. page 317

8. There are several other ways to accomplish this. We could use inner classes in Java.

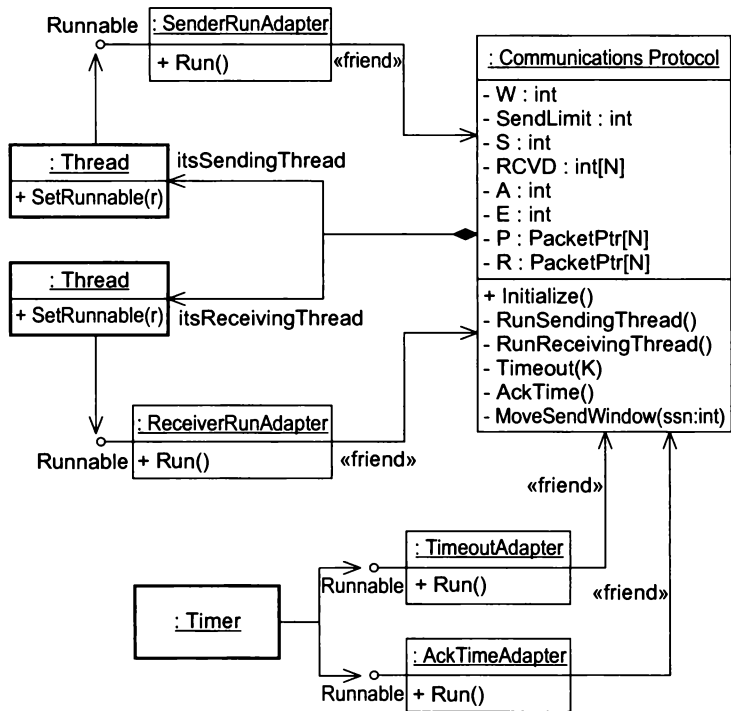
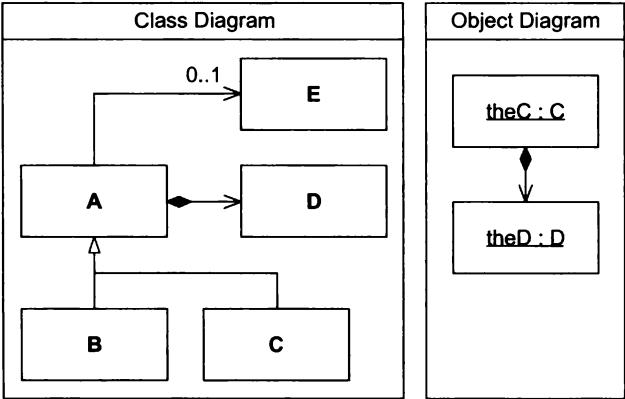


Figure B-8 Object Diagram: Just after protocol object has been initialized

Object Diagrams

Object diagrams depict the static relationships that exist between a set of objects at a particular instance in time. They differ from class diagrams in two ways. First, they depict *objects* instead of *classes* and the *links* between objects rather than the *relationships* between classes. Second, class diagrams show source code relationships and dependencies, whereas object diagrams show only those run-time relationships and dependencies that exist for the instant defined by the object diagram. Thus, the object diagram shows the objects and links that exist when the system is in a particular state.



(continues on next page)

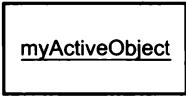
In the previous diagram, we see a class diagram and an object diagram that represent one possible state of the objects and links that derive from the classes and relationships in the class diagram. Notice that the objects are drawn in the same manner as they are in a sequence diagram. They are rectangles with underlined, two-component names. Notice also that the relationship on the object diagram is drawn the same way as on the class diagram.

A relationship between two objects is called a link. A link allows messages to flow in the direction of navigability. In this case, messages can flow from theC to theD. This link exists because there is a composition relationship between class A and class D, and because class C derives from class A. Thus, an instance of class C can have links that are derived from its base classes.

Notice also that the relationship between class A and class E is not represented on the object diagram. This is because the object diagram depicts a particular state of the system during which C objects are not associated with E objects.

Active Objects

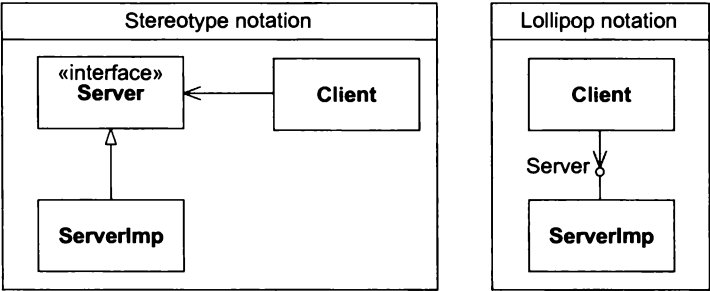
Active objects are objects that are responsible for a single thread of execution. The thread of execution does not need to be running inside the methods of the active object. Indeed, the active object typically calls out to other objects. The active object is simply the object in which the thread of execution originates. It is also the object that provides thread management interfaces such as `Terminate` and `Suspend`, and `ChangePriority`.



Active objects are drawn as ordinary objects, but with a bold outline. If the active object also owns other objects that execute within its thread of control, you can draw those objects inside the boundaries of the active object.

Interface Lollipops

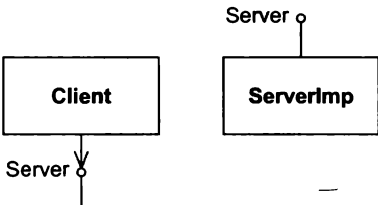
Interfaces can be shown as classes with the «interface» stereotype, or they may be shown with a special lollipop icon.



(continues on next page)

Both of the diagrams in this box have exactly the same interpretation. Instances of class `Client` make use of the `Server` interface. The `ServerImp` class implements the `Server` interface.

Either of the two relationships connected to the lollipop can be omitted, as shown in the following diagram:



The Initialization Process. The individual processing steps used to initialize the `CommunicationsProtocol` object are shown in Figure B-9. This is a collaboration diagram. (See “Collaboration Diagram” on page 504.

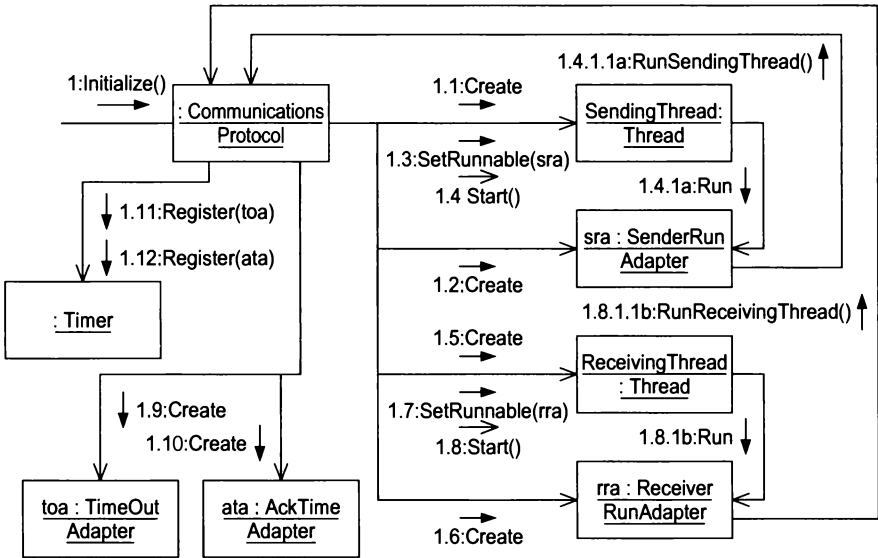


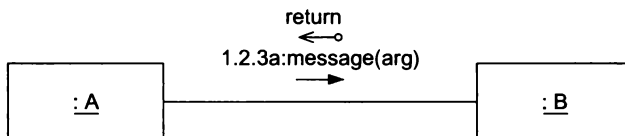
Figure B-9 Collaboration Diagram: Initializing the `CommunicationsProtocol` object

The initialization process begins at message number 1. The `CommunicationsProtocol` receives the `Initialize` message from some unknown source. It responds in messages 1.1 and 1.2 by creating the `SendingThread` object and its associated `SenderRunAdapter`. Then, in messages 1.3 and 1.4, it binds the adapter to the thread and starts the thread.

Notice that message 1.4 is asynchronous, so the initialization process continues with messages 1.5 through 1.8, which simply repeat the procedure for the creation of the `ReceivingThread`. Meanwhile, a separate thread of execution begins in message 1.4.1a, which invokes the `Run` method in the `SenderRunAdapter`. As a result, the adapter sends message 1.4.1.1a:RunSendingThread to the `CommunicationsProtocol` object. This starts the processing of the sending thread. A similar chain of events starts the receiving thread. Finally, messages 1.9 through 1.12 create the timer adapters and register them with the timer.

Collaboration Diagrams

Collaboration diagrams are similar to object diagrams except that they show how the state of the system evolves over time. The messages that are sent between the objects are shown, along with their arguments and return values. Each message is labeled with a sequence number to show the order in relation to the other messages.



Messages are drawn as small arrows placed near the link between two objects. The arrow points at the object that is receiving the message. The message is labeled with the name and sequence number of the message.

The sequence number is separated from the name of the message by a colon. The message name is followed by parentheses that contain a comma-separated list of the arguments to the message. The sequence number is a dot-separated list of numbers, followed by an optional thread identifier.

The numbers in the sequence number denote both the order of the message and its depth in the calling hierarchy. Message number 1 is the first message to be sent. If the procedure invoked by message 1 invokes two other messages, they will be numbered 1.1 and 1.2, respectively. Once they return and message number 1 completes, the next message will be number 2. By using this scheme of dots, it is possible to completely describe the order and nesting of the messages.

The thread identifier is the name of the thread that the message is executing within. If the thread identifier is omitted, it indicates that the message is executing in the unnamed thread. If message number 1.2 spawns a new thread named “t,” the first message of that new thread will be numbered 1.2.1t.

Return values and arguments can be shown by using the data token symbol (the little arrow with the circle on the end). Alternately, return values can be shown using assignment syntax in the message name as follows:

```
1.2.3 : c:=message(a,b)
```

In this case, the return value of “message” will be held in a variable named “c.”

→ A message that uses a filled arrowhead, as shown to the left, represents a synchronous function call. It does not return until all other synchronous messages invoked from its procedure are returned. This is the normal kind of message for C++, Smalltalk, Eiffel, or Java, etc.

→ The stick arrowhead shown to the left represents an asynchronous message. Such a message spawns a new thread of control to execute the invoked method and then returns immediately. Thus, the message returns before the method is executed. Messages that are sent by the method should have a thread identifier, since they are executing in a thread that differs from the invocation.

Race Conditions within the Protocol. The protocol, as described in Figure B-7, has a number of interesting *race conditions*. A race condition occurs when the order of two separate events cannot be predicted, yet the state of the system is sensitive to that order. The state of the system then depends on which event wins the race.

The programmer attempts to make sure that the system behaves properly regardless of the ordering of the events. However, race conditions are hard to identify. Undiscovered race conditions can lead to transient and difficult-to-diagnose errors.

As an example of a race condition, consider what happens when a packet is sent by the sending thread. (See Figure B-10.) This kind of diagram is called a message sequence chart. (See “Message Sequence Charts” on page 506.) The local sender sends packet S and starts a time-out. The remote receiver receives this packet and lets the remote sender know that S was received OK. The remote sender either sends an explicit ACK or piggybacks an ACK onto the next packet. The local receiver gets this ACK and kills the time-out.

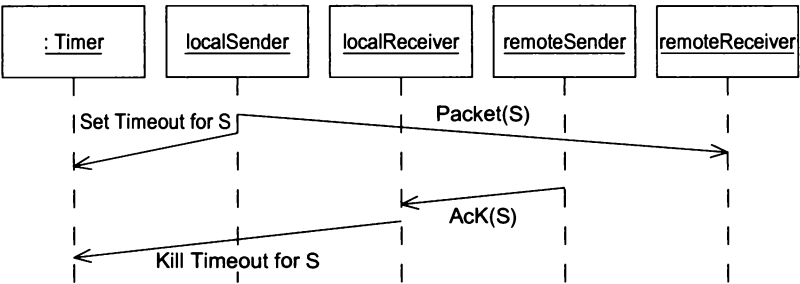


Figure B-10 Acknowledgment of a packet: Normal

Sometimes the ACK doesn’t make it back. In that case the time-out expires, and the packet is retransmitted. Figure B-11 shows what happens.

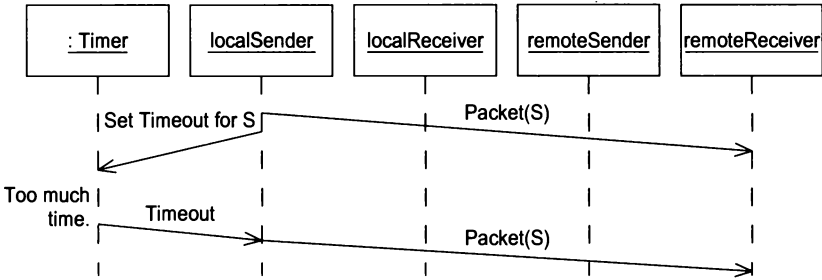


Figure B-11 Acknowledgment lost: Retransmission

A race condition exists between these two extremes. It is possible that the timer will expire just as the ACK is being sent. Figure B-12 shows the scenario. Note the crossed lines. They represent the race. Packet S has been sent and received just fine. Moreover, an ACK was transmitted back. However, the ACK arrived after the time-out had occurred. Thus, the packet gets retransmitted even though the ACK was received.

The logic of Figure B-7 handles this race properly. The remote receiver will realize that the second arrival of packet S is a duplicate and will discard it.

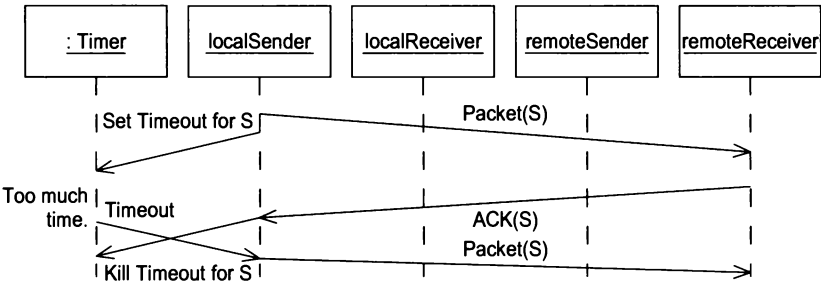


Figure B-12 ACK / Retransmission race condition

Message Sequence Charts

Message sequence charts are a special form of sequence diagrams. The primary difference is that the message arrows are angled downward to show that time can elapse between the sending and the receiving of a message. All the other parts of the sequence diagram may be present, including activations and sequence numbers.

The primary use of message sequence charts is to discover and document race conditions. These charts are very good at showing the relative timings of certain events and how two independent processes can have a different view of the order of events.

Consider Figure B-12. The Timer object thinks that the Timeout event occurs before the Kill Timeout event. However, the localSender perceives those two events in the opposite order.

This difference in the perception of the order of events can lead to logic flaws that are extremely sensitive to timing and very hard to reproduce and diagnose. Message sequence charts are a very nice tool for finding these situations before they wreak havoc in the field.

Conclusion

In this chapter, we have presented most of the dynamic modeling techniques of UML. We have seen state machines, activity diagrams, collaboration diagrams, and message sequence charts. We have also seen how these diagrams handle problems of single and multiple threads of control.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison-Wesley, 1995.

APPENDIX C

A Satire of Two Companies

"I've got a good mind to join a club and beat you over the head with it!"

—Rufus T. Firefly

Rufus, Inc. Project Kickoff

Your name is Bob. The date is January 3, 2001, and your head still aches from the recent millennial revelry. You are sitting in a conference room with several managers and a group of your peers. You are a project team leader. Your boss is there, and he has brought along all of his team leaders. His boss called the meeting.

"We have a new project to develop," says your boss's boss. Call him BB. The points in his hair are so long that they scrape the ceiling. Your boss's points are just starting to grow, but he eagerly awaits the day when he can leave Brylcream stains on the acoustic tiles. BB describes the essence of the new market they have identified and the product they want to develop to exploit this market.

"We must have this new project up and working by fourth quarter, October 1," BB demands. "Nothing is of higher priority; so we are cancelling your current project."

The reaction in the room is stunned silence. Months of work are simply going to be thrown away. Slowly, a murmur of objection begins to circulate around the conference table.

His points give off an evil green glow as BB meets the eyes of everyone in the room. One by one that insidious stare reduces each attendee to quivering lumps of protoplasm. It is clear that he will brook no discussion on this matter.

Once silence has been restored, BB says, "We need to begin immediately. How long will it take you to do the analysis?"

Rupert Industries Project: ~Alpha~

Your name is Robert. The date is January 3, 2001. The quiet hours spent with your family this holiday have left you refreshed and ready for work. You are sitting in a conference room with your team of professionals. The manager of the division called the meeting.

"We have some ideas for a new project," says the division manager. Call him Russ. He is a high-strung British chap with more energy than a fusion reactor. He is ambitious and driven, but he understands the value of a team.

Russ describes the essence of the new market opportunity the company has identified and introduces you to Jay, the marketing manager who is responsible for defining the products that will address it.

Addressing you, Jay says, "We'd like to start defining our first product offering as soon as possible. When can you and your team meet with me?"

You reply, "We'll be done with the current iteration of our project this Friday. We can spare a few hours for you between now and then. After that, we'll take a few people from the team and dedicate them to you. We'll begin hiring their replacements and the new people for your team immediately."

"Great," says Russ, "But I want you to understand that it is critical that we have something to exhibit at the trade show coming up this

You raise your hand. Your boss tries to stop you, but his spitwad misses you and you are unaware of his efforts.

"Sir, we can't tell you how long the analysis will take until we have some requirements."

"The requirements document won't be ready for three or four weeks," BB says, his points vibrating with frustration. "So, *pretend* that you have the requirements in front of you now. How long will you require for analysis?"

No one breathes. Everyone looks around at everybody else to see if they have some idea.

"If analysis takes any longer than April 1, then we have a problem. Can you finish the analysis by then?"

Your boss visibly gathers his courage, building to the ejaculation, "We'll find a way, Sir!" His points grow 3 mm, and your headache increases by two Tylenols.

"Good." BB smiles. "Now, how long will it take to do the design?"

"Sir," you say. Your boss visibly pales. He is clearly worried that his 3 millimeters are at risk. "Without an analysis, it will not be possible to tell you how long design will take."

BB's expression shifts beyond austere. "PRE-TEND you have the analysis already!" he says, while fixing you with his vacant beady little eyes. "How long will it take you to do the design?"

Two Tylenols are not going to cut it. Your boss, in a desperate attempt to save his new growth, babbles, "Well, sir, with only six months left to complete the project, design had better take no longer than three months."

"I'm glad you agree, Smithers!" BB says, beaming. Your boss relaxes. He knows his points are secure. After a while, he starts lightly humming the Brylcream jingle.

BB continues, "So, analysis will be complete by April 1, design will be complete by July 1, and that gives you three months to implement the project. This meeting is an example of how well our new consensus and empowerment policies are working. Now, get out there and start working. I'll expect to see TQM plans and QIT assignments on my desk by next week. Oh, and don't forget that your cross-functional team meetings and reports will be needed for next month's quality audit."

"Forget the Tylenol," you think to yourself as you return to your cubicle. "I need bourbon."

July. If we can't be there with something significant, we'll lose the opportunity."

"I understand," you reply. "I don't yet know what it is that you have in mind, but I'm sure we can have something by July. I just can't tell you what that something will be right now. In any case, you and Jay are going to have complete control over what we developers do, so you can rest assured that by July you'll have the most important things that can be accomplished in that time ready to exhibit."

Russ nods in satisfaction. He knows how this works. Your team has always kept him advised and allowed him to steer their development. He has the utmost confidence that your team will work on the most important things first and that they will produce a high-quality product.

~ ~ ~

"So Robert," says Jay at the first meeting, "How does your team feel about being split up?"

"We'll miss working with each other," you answer, "but some of us were getting pretty tired of that last project and are looking forward to a change. So, what are you guys cooking up?"

Jay beams. "You know how much trouble our customers currently have..." And he spends a half hour or so describing the problem and possible solution.

"OK, wait a second" you respond. "I need to be clear about this." And so you and Jay talk about how this system might work. Some of Jay's ideas aren't fully formed. You suggest possible solutions. He likes some of them. You continue discussing.

During the discussion, as each new topic is addressed, Jay writes user-story cards. Each card represents something that the new system has to do. The cards accumulate on the table and are spread out in front of you. Both you and Jay point at them, pick them up, and make notes on them as you discuss the stories. The cards are powerful mnemonic devices that you can use to represent complex ideas that are barely formed.

At the end of the meeting you say, "OK, I've got a general idea of what you want. I'm going to talk to the team about it. I imagine there are some experiments they'll want to run with various data-

Visibly excited, your boss comes over to you and says, “Gosh, what a great meeting. I think we’re really going to do some world shaking with this project.” You nod in agreement, too disgusted to do anything else.

“Oh,” your boss continues, “I almost forgot.” He hands you a 30-page document. “Remember that the SEI are coming to do an evaluation next week. This is the evaluation guide. You need to read through it, memorize it, and then shred it. It tells you how to answer any questions that the SEI auditors ask you. It also tells you what parts of the building you are allowed to take them to and what parts to avoid. We are determined to be a CMM level 3 organization by June!”

* * *

You and your peers start working on the analysis of the new project. This is difficult because you have no requirements. But, from the 10-minute introduction given by BB on that fateful morning, you have some idea of what the product is supposed to do.

Corporate process demands that you begin by creating a use-case document. You and your team begin enumerating use cases and drawing oval and stick diagrams.

Philosophical debates break out among the team. There is disagreement as to whether certain use cases should be connected with <<extends>> or <<includes>> relationships. Competing models are created, but nobody knows how to evaluate them. The debate continues, effectively paralyzing progress.

After a week, somebody finds the iceberg.com Web site that recommends disposing entirely of <<extends>> and <<includes>> and replacing them with <<precedes>> and <<uses>>. The documents on this Web site, authored by Don Sengroix, describe a method known as Stalwart analysis, which claims to be a step-by-step method for translating use cases into design diagrams.

More competing use-case models are created using this new scheme; but again, nobody agrees on how to evaluate them. And the thrashing continues.

More and more, the use-case meetings are driven by emotion rather than reason. If it weren’t for the fact that you don’t have requirements, you’d be pretty upset by the lack of progress you are making.

The requirements document arrives on the 15th of February. And then again on the 20th, 25th, and every week thereafter. Each new version contradicts the previous. Clearly, the marketing folks who are writing the

base structures and presentation formats. Next time we meet, it’ll be as a group, and we’ll start identifying the most important features of the system.

A week later, your nascent team meets with Jay. They spread the existing user-story cards out on the table and begin to get into some of the details of the system.

The meeting is very dynamic. Jay presents the stories in the order of their importance. There is much discussion about each one. The developers are concerned about keeping the stories small enough to estimate and test. So they continually ask Jay to split one story into several smaller stories. Jay is concerned that each story has a clear business value and priority, so as he splits them, he makes sure this stays true.

The stories accumulate on the table. Jay writes them, but the developers make notes on them as needed. Nobody tries to capture everything that is said. The cards are not meant to capture everything; they are just reminders of the conversation.

As the developers become more comfortable with the stories, they begin writing estimates on them. These estimates are crude and budgetary, but they give Jay an idea of what the story will cost.

At the end of the meeting, it is clear that there are many more stories that could be discussed. It is also clear that the most important stories have been addressed and that they represent several months’ worth of work. Jay closes the meeting by taking the cards with him and promising to have a proposal for the first release in the morning.

~ ~ ~

The next morning, you reconvene the meeting. Jay chooses five cards and places them on the table.

“According to your estimates, these cards represent about fifty points worth of work. The last iteration of the previous project managed to get fifty points done in three weeks. If we can get these five stories done in three weeks, we’ll be able to demonstrate them to Russ. That will make him feel very comfortable about our progress.”

Jay is pushing it. The sheepish look on his face lets you know that he knows it too. You reply,

requirements, empowered though they might be, are not finding consensus.

At the same time, several new competing use-case templates have been proposed by the various team members. Each presents its own particularly creative way of delaying progress. The debates rage on.

On March 1, Percival Putrignence, the process proctor, succeeds in integrating all the competing use-case forms and templates into a single, all-encompassing form. Just the blank form is 15 pages long. He has managed to include every field that appeared on all the competing templates. He also presents a 159-page document describing how to fill out the use-case form. All current use cases must be rewritten according to the new standard.

You marvel to yourself that it now requires 15 pages of fill-in-the-blank and essay questions to answer the question, “What should the system do when the user hits return?”

The corporate process (authored by L. E. Ott, famed author of “Holistic analysis: A progressive dialectic for software engineers”) insists that you discover all primary use cases, 87% of all secondary use cases, and 36.274% of all tertiary use cases before you can complete analysis and enter the design phase. You have no idea what a tertiary use case is. So in an attempt to meet this requirement, you try to get your use-case document reviewed by the marketing department. Maybe *they* know what a tertiary use case is.

Unfortunately, the marketing folks are too busy with sales support to talk to you. Indeed, since the project started, you have not been able to get a single meeting with marketing. The best they have been able to do is provide a never-ending stream of changing and contradictory requirements documents.

While one team has been spinning endlessly on the use-case document, another has been working out the domain model. Endless variations of UML documents are pouring out of this team. Every week, the model is reworked. The team members can’t decide on whether to use <<interfaces>> or <<types>> in the model. A huge disagreement has been raging on the proper syntax and application of OCL. Others in the team just got back from a five-day class on “catabolism” and have been producing incredibly detailed and arcane diagrams that nobody else can fathom.

On March 27, with one week to go before analysis is to be complete, you have produced a sea of documents

“Jay, this is a new team, working on a new project. It’s a bit presumptuous to expect that our velocity will be the same as the previous team’s. However, I met with the team yesterday afternoon, and we all agreed that our initial velocity should, in fact, be set to fifty story points for every three weeks. So you’ve lucked out on this one.”

“Just remember,” you continue, “that the story estimates and the velocity are very tentative at this point. We’ll learn more when we plan the iteration and even more when we implement it.”

Jay looks over his glasses at you as if to say, “Who’s the boss around here anyway?” Then he smiles and says, “Yeah, don’t worry, I know the drill by now.”

Jay then puts 15 more cards on the table. He says, “If we can get all these cards done by the end of March, we can turn the system over to our beta test customers. And we’ll get good feedback from them.”

You reply, “OK, so we’ve got our first iteration defined, and we have the stories for the next three iterations after that. These four iterations will make our first release.”

“So,” says Jay, “Can you really do these five stories in the next three weeks?”

“I don’t know for sure Jay,” you reply, “Let’s break them down into tasks and see what we get.”

*So Jay, you, and your team spend the next several hours taking each of the five stories that Jay chose for the first iteration and breaking it down into small tasks. The developers quickly realize that some of the tasks can be shared between stories and that other tasks have commonalities that can probably be taken advantage of. It is clear that potential designs are popping into the developers’ heads. From time to time, they form little discussion knots and scribble **UML** diagrams on some cards.*

Soon, the whiteboard is filled with the tasks that, once completed, will implement the five stories for this iteration. You start the sign-up process by saying, “OK, let’s sign up for these tasks.”

“I’ll take the initial database generation,” says Pete, “That’s what I did on the last project, and this doesn’t look very different. I estimate it at two days.”

and diagrams, but you are no closer to a cogent analysis of the problem than you were on January 3.

* * *

And then, a miracle happens.

* * *

On Saturday, April 1, you check your e-mail from home. You see a memo from your boss to BB. It states unequivocally that you are done with the analysis!

You phone your boss and complain, "How could you have told BB that we were done with the analysis?"

"Have you looked at a calendar lately?" he responds, "It's April 1!"

The irony of that date does not escape you. "But we have so much more to think about. So much more to analyze! We haven't even decided whether to use <<extends>> or <<precedes>>!"

"Where is your evidence that you are not done?" inquires your boss impatiently.

"Whaaa...."

But he cuts you off: "Analysis can go on forever, it has to be stopped at some point. And since this is the date it was scheduled to stop, it has been stopped now. On Monday I want you to gather up all existing analysis materials and put them into a public folder. Release that folder to Percival so that he can log it in the CM system by Monday afternoon. Then get busy and start designing."

As you hang up the phone, you begin to consider the benefits of keeping a bottle of bourbon in your bottom desk drawer.

* * *

They threw a party to celebrate the on-time completion of the analysis phase. BB gave a colon stirring speech on empowerment. And your boss, another 3 mm taller, congratulated his team on the incredible show of unity and teamwork. Finally, the CIO takes the stage and tells everyone that the SEI audit went very well, and he thanks everyone for studying and shredding the evaluation guides that were passed out. Level 3 now seems assured and will be awarded by June.

(Scuttlebutt has it that managers at the level of BB and above are to receive significant bonuses once the SEI awards level 3.)

As the weeks flow by, you and your team work on the design of the system. Of course you find that the analysis that the design is supposedly based upon is flawed... no, useless... no, worse than useless. But when you tell your boss that you need to go back and work

"OK, well then I'll take the login screen," says Joe.

"Aw darn," says Elmo, the junior member of the team, "I've never done a GUI, and I kinda wanted to try that one."

"Ah, the impatience of youth," Joe says sagely, with a wink in your direction, "You can assist me with it, young Jedi." To Jay, he says, "I think it'll take me about three days."

One by one, the developers sign up for tasks and estimate them. Both you and Jay know that it is better to let the developers volunteer for tasks than it is to assign the tasks to them. You also know full well that you daren't challenge any of the developers' estimates. You know these guys, and you trust them. You know they are going to do the very best they can.

The developers know that they can't sign up for more than they finished in the last iteration they worked on. Once each developer has filled his schedule for the iteration, he stops signing up for tasks.

Eventually, all the developers have stopped signing up for tasks. But, of course, there are still tasks left on the board.

"I was worried that that might happen," you say. "OK, there's only one thing to do, Jay. We've got too much to do in this iteration. What stories or tasks can we remove?"

Jay sighs. He knows that this is the only option. Working overtime at the beginning of a project is insane, and projects where he's tried it have not fared well.

So Jay starts to remove the least important functionality. "Well, we really don't need the login screen just yet. We can simply start the system in the logged-in state."

"Rats!" cries Elmo. "I really wanted to do that."

"Patience, Grasshopper," says Joe. "Those who wait for the bees to leave the hive will not have lips too swollen to relish the honey."

Elmo looks confused.

Everyone looks confused.

"So...", Jay continues, "I think we can also do away with..."

some more on the analysis to shore up its weaker sections, he simply states, “The analysis phase is over. The only allowable activity is design. Now get back to it.”

So, you and your team hack the design as best you can, unsure of whether the requirements have been properly analyzed or not. Of course it really doesn't matter much since the requirements document is still thrashing with weekly revisions, and the marketing department still refuses to meet with you.

The design is a nightmare. Your boss recently misread a book named *The Finish Line* in which the author, Mark DeThomaso, blithely suggested that design documents should be taken down to code-level detail.

“If we are going to be working at that level of detail,” you ask, “why don't we just write the code instead?”

“Because then you wouldn't be designing, of course. And the only allowable activity in the design phase is design!”

“Besides,” he continues, “we have just purchased a company-wide license for Dandelion! This tool enables “round-the-horn engineering!” You are to transfer all design diagrams into this tool. It will automatically generate our code for us! It will also keep the design diagrams in sync with the code!”

Your boss hands you a brightly colored, shrink-wrapped box containing the Dandelion distribution. You accept it numbly and shamble off to your cubicle. Twelve hours, eight crashes, a disk reformatting, and eight shots of 151 later, you finally have the tool installed on your server. You consider the week your team will lose while attending Dandelion training. Then you smile and think, “Any week I'm not here is a good week.”

Design diagram after design diagram is created by your team. Dandelion makes it very hard to draw these diagrams. There are dozens and dozens of deeply nested dialog boxes with funny text fields and check boxes that must all be filled in correctly. And then there's the problem of moving classes between packages...

At first, these diagrams are driven from the use cases. But the requirements are changing so often that the use cases rapidly become meaningless.

Debates rage about whether VISITOR or DECORATOR design patterns should be employed. One developer refuses to use VISITOR in any form, claiming that it's not a properly object-oriented construct. Another refuses to use multiple inheritance since it is the spawn of the devil.

And so, bit by bit, the list of tasks shrinks. Developers who lose a task sign up for one of the remaining ones.

The negotiation is not painless. Several times, Jay exhibits obvious frustration and impatience. Once, when tensions are especially high, Elmo volunteers to “Work extra hard to make up some of the missing time.” You are about to correct him when, fortunately, Joe looks him in the eye and says, “When once you proceed down the dark path, forever will it dominate your destiny.”

In the end, an iteration acceptable to Jay is reached. It's not what Jay wanted. Indeed, it is significantly less. But it's something the team feels that they can achieve in the next three weeks. And, after all, it still addresses the most important things that Jay wanted in the iteration.

“So, Jay,” you say when things have quieted down a bit. “When can we expect acceptance tests from you?”

Jay sighs. This is the other side of the coin. For every story the development team implements, Jay must supply a suite of acceptance tests that prove that they work. And the team needs these long before the end of the iteration, since they will certainly point out differences in the way Jay and the developers imagine the system's behavior.

“I'll get you some example test scripts today,” Jay promises. “I'll add to them every day after that. You'll have the entire suite by the middle of the iteration.”

~ ~ ~

The iteration begins on Monday morning with a flurry of CRC sessions. By midmorning, all the developers have assembled into pairs and are rapidly coding away.

“And now, my young apprentice,” Joe says to Elmo, “you shall learn the mysteries of test-first design!”

“Wow, that sounds pretty rad,” Elmo replies. “How do you do it?”

Joe beams. It's clear that he has been anticipating this moment. “Laddy-buck, what does the code do right now?”

“Huh?” replies Elmo, “It doesn't do anything at all, there is no code.”

Review meetings rapidly degenerate into debates about the meaning of object orientation, the definition of analysis vs. design, or when to use aggregation vs. association.

Midway through the design cycle, the marketing folks announce that they have rethought the focus of the system. Their new requirements document is completely restructured. They have eliminated several major feature areas and replaced them with feature areas that they anticipate customer surveys will show to be more appropriate.

You tell your boss that these changes mean that you need to reanalyze and redesign much of the system. But he says, "The analysis phase is over. The only allowable activity is design. Now get back to it."

You suggest that it might be better to create a simple prototype to show to the marketing folks, and even some potential customers. But your boss says, "The analysis phase is over. The only allowable activity is design. Now get back to it."

Hack, hack, hack, hack. You try to create some kind of a design document that might actually reflect the new requirements documents. However, the revolution of the requirements has not caused them to stop thrashing. Indeed, if anything, the wild oscillations of the requirements document have only increased in frequency and amplitude. You slog your way through them.

On June 15, the Dandelion database gets corrupted. Apparently the corruption has been progressive. Small errors in the database accumulated over the months into bigger and bigger errors. Eventually the CASE tool just stopped working. Of course the slowly encroaching corruption is present on all the backups.

Calls to the Dandelion technical support line go unanswered for several days. Finally you receive a brief e-mail from Dandelion, informing you that this is a known problem, and the solution is to purchase the new version (which they promise will be ready some time next quarter) and then reenter all the diagrams by hand.

* * *

Then, on July 1, another miracle happens! You are done with the design!

Rather than go to your boss and complain, you stock your middle desk drawer with some vodka.

* * *

They threw a party to celebrate the on-time completion of the design phase and their graduation to CMM

"So, consider our task. Can you think of something the code should do?"

"Sure." Elmo says with youthful assurance, "First, it should connect to the database."

"And thereupon, what must needs be required to connecteth the database?"

"You sure talk weird," laughs Elmo. "I think we'd have to get the database object from some registry and call the Connect() method."

"Ah. Astute young wizard. Thou perceivest correctly that we requireth an object within which we can cacheth the database object."

"Is 'cacheth' really a word?"

"It is when I say it! So, what test can we write that we know the database registry should pass?"

Elmo sighs. He knows he'll just have to play along. "We should be able to create a database object and pass it to the registry in a Store() method. And then we should be able to pull it out of the registry with a Get() method and make sure it's the same object."

"Oh, well said, my prepubescent sprite!"

"Hey!"

"So, now, let's write a test function that proves your case."

"But shouldn't we write the database object and registry object first?"

"Ah, you've much to learn my young, impatient one. Just write the test first."

"But it won't even compile!"

"Are you sure? What if it did?"

"Uh..."

"Just write the test, Elmo. Trust me."

And so Joe, Elmo, and all the other developers begin to code their tasks, one test case at a time. The room in which they work is a-buzz with the conversations between the pairs. The murmur is punctuated by an occasional high-five when a pair manages to finish a task or a difficult test case.

As development proceeds, the developers change partners once or twice a day. Each developer gets to see what all the others are doing, and so knowledge of the code spreads generally throughout the team.

Whenever a pair finishes something significant, whether a whole task or just an important

level 3. This time you find BB's speech so stirring that you have to use the restroom before it begins.

There are new banners and plaques all over your workplace. They show pictures of eagles and mountain climbers, and they talk about teamwork and empowerment. They read better after a few scotches. That reminds you that you need to clear out your file cabinet to make room for the brandy.

You and your team begin to code. But you rapidly discover that the design is lacking in some significant areas. Actually it's lacking any significance at all. You convene a design session in one of the conference rooms to try to work through some of the nastier problems. But your boss catches you at it and disbands the meeting saying, "The design phase is over. The only allowable activity is coding. Now get back to it."

The code generated by Dandelion is really hideous. It turns out that you and your team were using association and aggregation the wrong way after all. All the generated code has to be edited to correct these flaws. Editing this code is extremely difficult because it has been instrumented with ugly comment blocks that have special syntax that Dandelion needs in order to keep the diagrams in sync with the code. If you accidentally alter one of these comments, then the diagrams will be regenerated incorrectly. It turns out that "round-the-horn engineering" requires an awful lot of effort.

The more you try to keep the code compatible with Dandelion, the more errors Dandelion generates. In the end, you give up and decide to keep the diagrams up to date manually. A second later, you decide there's no point in keeping the diagrams up to date at all. Besides, who has time?

Your boss hires a consultant to build tools to count the number of lines of code that are being produced. He puts a big thermometer graph on the wall with the number 1,000,000 on the top. Every day he extends the red line to show how many lines have been added.

Three days after the thermometer appears on the wall, your boss stops you in the hall. "That graph isn't growing fast enough. We need to have a million lines done by October 1."

"We aren't even sh-sh-sure that the proshect will require a m-million linezh," you blather.

"We have to have a million lines done by October 1," your boss reiterates. His points have grown again, and

part of a task, they integrate what they have with the rest of the system. Thus, the code base grows daily, and integration difficulties are minimized.

The developers communicate with Jay on a daily basis. They go to him whenever they have a question about the functionality of the system or the interpretation of an acceptance test case.

Jay, good as his word, supplies the team with a steady stream of acceptance-test scripts. The team reads these carefully and thereby gains a much better understanding of what Jay expects the system to do.

By the beginning of the second week, there is enough functionality to demonstrate to Jay. Jay watches eagerly as the demonstration passes test case after test case.

"This is really cool," Jay says as the demonstration finally ends. "But this doesn't seem like one-third of the tasks. Is your velocity slower than anticipated?"

You grimace. You've been waiting for a good time to mention this to Jay, but now Jay is forcing the issue.

"Yes, unfortunately we are going slower than we had expected. The new application server we are using is turning out to be a pain to configure. Also, it takes forever to reboot, and we have to reboot it whenever we make even the slightest change to its configuration."

Jay eyes you with suspicion. The stress of last Monday's negotiations has still not entirely dissipated. He says, "And what does this mean to our schedule? We can't slip it again, we just can't. Russ will have a fit! He'll haul us all into the woodshed and ream us some new ones."

You look Jay right in the eyes. There's no pleasant way to give someone news like this. So you just blurt out, "Look, if things keep going like their going, then we're not going to be done with everything by next Friday! Now it's possible that we'll figure out a way to go faster. But, frankly, I wouldn't depend on that. You should start thinking about one or two tasks that could be removed from the iteration without ruining the demonstration for Russ. Come hell or high water, we are going to give that demonstration on Friday, and I don't think you want us to choose which tasks to omit."

the Grecian formula he uses on them creates an aura of authority and competence. “Are you sure your comment blocks are big enough?”

Then, in a flash of managerial insight he says, “I have it! I want you to institute a new policy among the engineers. No line of code is to be longer than 20 characters. Any such line must be split into two or more—preferably more. All existing code needs to be reworked to this standard. That’ll get our line count up!”

You decide not to tell him that this will require two unscheduled man-months. You decide not to tell him anything at all. You decide that intravenous injections of pure ethanol are the only solution. You make the appropriate arrangements.

Hack, hack, hack, and hack. You and your team madly code away. By August 1, your boss, frowning at the thermometer on the wall institutes a mandatory 50-hour workweek.

Hack, hack, hack, and hack. By September 1, the thermometer is at 1.2 million lines, and your boss asks you to write a report describing why you exceeded the coding budget by 20%. He institutes mandatory Saturdays and demands that the project be brought back down to a million lines. You start a campaign of remerging lines.

Hack, hack, hack, and hack. Tempers are flaring; people are quitting; QA is raining trouble reports down on you. Customers are demanding installation and user manuals; salesmen are demanding advance demonstrations for special customers; the requirements document is still thrashing; the marketing folks are complaining that the product isn’t anything like they specified, and the liquor store won’t accept your credit card anymore. Something has to give. On September 15, BB calls a meeting.

As he enters the room, his points are emitting clouds of steam. When he speaks, the bass overtones of his carefully manicured voice cause the pit of your stomach to roll over. “The QA manager has told me that this project has less than 50% of the required features implemented. He has also informed me that the system crashes all the time, yields wrong results, and is hideously slow. He has also complained that he cannot keep up with the continuous train of daily releases, each more buggy than the last!”

He stops for a few seconds, visibly trying to compose himself. “The QA manager estimates that, at this

“Aw, for Pete’s sake!” Jay barely manages to stifle yelling his last word as he stalks away shaking his head.

Not for the first time, you say to yourself, “Nobody ever promised me project management would be easy.” You are pretty sure it won’t be the last time either.

~ ~ ~

Actually, things go a bit better than you had hoped. The team does, in fact, have to drop one task from the iteration, but Jay chooses wisely, and the demonstration for Russ goes without a hitch.

Russ is not impressed with the progress, but neither is he dismayed. He simply says, “This is pretty good. But remember, we have to be able to demonstrate this system at the trade show in July, and at this rate it doesn’t look like you’ll have all that much to show.”

Jay, whose attitude has improved dramatically with the completion of the iteration, responds to Russ by saying, “Russ, this team is working hard and well. When July comes around, I am confident that we’ll have something significant to demonstrate. It won’t be everything, and some of it may be smoke and mirrors, but we’ll have something.”

Painful though the last iteration was, it calibrated your velocity numbers. The next iteration goes much better. Not because your team gets more done than in the last iteration, but simply because they don’t have to remove any tasks or stories in the middle of the iteration.

By the start of the fourth iteration, a natural rhythm is established. Jay, you, and the team know exactly what to expect from each other. The team is running hard, but the pace is sustainable. You are confident that the team can keep up this pace for a year or more.

The number of surprises in the schedule diminishes to near zero; however, the number of surprises in the requirements does not. Jay and Russ frequently look over the growing system and make recommendations or changes to the existing functionality. But all parties realize that these changes take time and must be scheduled. So the changes do not cause anyone’s expectations to be violated.